UNIVERSIDAD TECNOLÓGICA DEL VALLE DE TOLUCA

INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN E INNOVACIÓN DIGITAL CON TSU EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

ASIGNATURA:

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

NOMBRE DEL DOCENTE:

MGTI. HÉCTOR VELÁZQUEZ ESTRADA

“Manual técnico del Sistema LumaIA ”

Versión 2.0

PRESENTA:

222410869 – JUÁREZ GONZÁLEZ IRAIS
222410849 – MENDOZA SÁNCHEZ ZUHEIDI
222410893 – NOYOLA SANABRIA JAREDTZI AIDE
222411200 – ROBLEDO PADILLA DORIAN BRISA
222411167 – SALINAS ARANDA OSVALDO

GRUPO: ITIID-D71

SANTA MARÍA ATARASQUILLO, LERMA, ESTADO DE MÉXICO, 27 DE OCTUBRE DE 2026

Control de versiones

Versión	Fecha	Descripción del cambio	Autor(es)
1.0	28/09/2026	Versión inicial bajo el estándar IEEE 830: arquitectura general, modelo de datos, diagramas (casos de uso, ER, diccionario de datos, clases), guía de instalación, configuración, despliegue y resolución de problemas.	Juárez González Irais Mendoza Sánchez Zuheidi Noyola Sanabria Jaredtzi Aide Robledo Padilla Dorian Brisa Salinas Aranda Osvaldo
2.0	27/10/2026	Se alinea el manual con el código fuente real de los tres repositorios (app móvil Expo, backend NestJS + GraphQL + MongoDB local con Compass y MongoDB Atlas en la nube, y motor IA FastAPI/Ollama): API GraphQL documentada, estructura de carpetas, configuración, pruebas ejecutadas al motor IA, hallazgos de seguridad, limitaciones, anexos y lista de verificación.	Juárez González Irais Mendoza Sánchez Zuheidi Noyola Sanabria Jaredtzi Aide Robledo Padilla Dorian Brisa Salinas Aranda Osvaldo

Estado del documento: vigente. **Clasificación:** uso académico. **Responsable de mantenimiento del documento:** equipo de desarrollo LumaIA (ITIID-D71).

Declaración de derechos de autor

© 2026 Jaredtzi Aide Noyola Sanabria, Zuheidi Mendoza Sánchez, Osvaldo Salinas Aranda, Irais Juárez González y Dorian Brisa Robledo Padilla. Todos los derechos reservados.

El presente manual fue elaborado con fines educativos y académicos y se encuentra protegido conforme a la Ley Federal del Derecho de Autor (LFDA) y los tratados internacionales en la materia. La estructura, redacción, organización y contenido desarrollado específicamente para este manual son de autoría propia de sus creadores. Queda prohibida su reproducción, modificación, distribución o publicación total o parcial con fines distintos a los académicos sin autorización previa y por escrito de sus autores.

Los nombres, logotipos, programas, lenguajes de programación, sistemas de bases de datos y demás recursos tecnológicos de terceros mencionados conservan sus respectivos derechos de propiedad intelectual. Su mención se realiza conforme al derecho de cita con fines educativos, informativos y de referencia (artículo 148 de la LFDA).

Índice

1. Introducción y glosario 6
1.1 Propósito del manual 6
1.2 Alcance 6
1.3 Conocimientos previos recomendados 6
1.4 Convenciones del documento 7
1.5 Glosario 7
2. Descripción general y requisitos funcionales y no funcionales 9
2.1 Descripción general del sistema 9
2.2 Actores del sistema 9
2.3 Marco PEAS del agente Luma 9
2.4 Variables proposicionales 10
2.5 Tablas de verdad 10
2.6 Requisitos funcionales 18
2.7 Requisitos no funcionales 20
2.8 Reglas de negocio 20
2.9 Historias de usuario 21
3. Arquitectura y tecnologías 22
3.1 Estilo arquitectónico 22
3.2 Componentes y comunicación 22
3.3 Flujo de una consulta al asistente 22
3.4 Pila tecnológica 23
3.5 Lenguajes utilizados 24
3.6 Puertos y protocolos 24
3.7 Patrones y estándares de codificación 24
3.8 Identidad visual y accesibilidad 25
3.9 Técnicas de búsqueda, distancia y optimización de IA 25
4. Instalación de paqueterías requeridas 33
4.1 Herramientas base que se asumen instaladas 33
4.2 Paquetes del backend por requisito 33
4.3 Paquetes de la aplicación móvil por requisito 34
4.4 Paquetes del motor IA 34
4.5 Elementos adicionales exigidos por los requisitos 35
4.6 Verificación rápida 35
5. Estructura del proyecto 36
5.1 Repositorios 36

5.2 Backend (lumaia-backend) 36
5.3 Aplicación móvil (lumaia-mobile) 37
5.4 Motor de inferencia (chatbot-lumaia) 37
6. Configuración 38
6.1 Variables de entorno del backend 38
6.2 Configuración del servidor GraphQL 38
6.3 Configuración de la aplicación móvil 39
6.4 Configuración del motor IA y de Ollama 39
6.5 Base de datos 39
6.6 Herramientas de calidad de código 40
7. Modelo de datos 41
7.1 Enfoque 41
7.2 Relaciones entre colecciones 41
7.3 Diccionario de datos 41
7.4 Enumeraciones 43
7.5 Índices 43
7.6 Datos de prueba 44
8. Módulos y API 45
8.1 Módulos del backend 45
8.2 Endpoint y forma de consumo 45
8.3 Queries (lectura) 45
8.4 Mutations (escritura) 46
8.5 Ejemplos de operaciones 46
8.6 API del motor de inferencia (FastAPI) 47
8.7 Lógica del AsistenteService 47
8.8 Capa de servicios de la aplicación 48
8.9 Navegación y pantallas 48
9. Pruebas 50
9.1 Estrategia de pruebas 50
9.2 Comandos 50
9.3 Pruebas automatizadas existentes 50
9.4 Pruebas ejecutadas al motor IA 51
9.5 Casos de prueba de la API GraphQL 51
9.6 Casos de prueba funcionales en la app 52
10. Despliegue y mantenimiento 53
10.1 Despliegue en entorno de desarrollo 53
10.2 Despliegue en producción 53

10.3 Mantenimiento preventivo 54
10.4 Procedimiento para agregar una funcionalidad 54
10.5 Control de versiones del código 54
11. Seguridad 56
11.1 Controles implementados 56
11.2 Hallazgos de la revisión de código 56
11.3 Consideraciones éticas y legales 57
12. Solución de problemas 58
13. Limitaciones y trabajo futuro 59
13.1 Limitaciones actuales 59
13.2 Trabajo futuro 59
Referencias 61
Anexos 62
Anexo A. Esquema GraphQL 62
Anexo B. Datos de prueba (database/seed.js) 64
Anexo C. Guía rápida de comandos 65
Lista de verificación antes de entregar 66

1. Introducción y glosario

1.1 Propósito del manual

Este manual técnico documenta el diseño interno, la configuración y la operación del sistema **LumaIA**, una aplicación multiplataforma que ayuda a las personas adultas mayores a organizar sus medicamentos, citas médicas y notas personales, con el apoyo de un asistente conversacional llamado **Luma** y de una red de contactos de emergencia.

Está dirigido a la persona o equipo que **dará mantenimiento al sistema**: corregir errores, agregar funcionalidades, actualizar dependencias o desplegarlo en un nuevo servidor. Por ello describe el sistema tal como está implementado en el código fuente de la versión 2.0 y señala con claridad qué funciones están completas, cuáles son parciales y cuáles están pendientes.

1.2 Alcance

El manual cubre los tres componentes de software que integran el sistema:

Tabla 1. *Componentes cubiertos por el manual*

Componente	Repositorio / paquete	Tecnología principal
Aplicación móvil (frontend)	`LumaIA_Frontend_Movil` (paquete `lumaia-mobile`)	Expo SDK 57, React Native 0.86, TypeScript
API y lógica de negocio (backend)	`LumaIA_Backend_2.0` (paquete `lumaia-backend`)	NestJS 11, GraphQL (Apollo), Mongoose, MongoDB
Motor de inteligencia artificial	`chatbot-lumaia` (servicio `LumaIA Engine API`)	Python, FastAPI, Pydantic; alternativa local con Ollama (Llama 3.2)

Quedan fuera del alcance la capacitación del usuario final (adulto mayor o cuidador), que corresponde al Manual de Usuario, y el proceso completo de instalación de sistemas operativos o herramientas de propósito general (ver sección 4).

1.3 Conocimientos previos recomendados

- TypeScript y JavaScript moderno (ES2021).
- React Native con Expo, React Navigation y TanStack React Query.
- NestJS (módulos, inyección de dependencias, resolvers) y GraphQL en modalidad *code-first*.
- MongoDB y el ODM Mongoose (esquemas, índices, ObjectId).
- Python 3.10 o superior y FastAPI para el motor de inferencia.
- Nociones de agentes inteligentes (marco PEAS) y lógica proposicional, base teórica de la asignatura.

1.4 Convenciones del documento

Tabla 2. *Convenciones tipográficas y de estado*

Convención	Significado
`texto monoespaciado`	Nombres de archivos, carpetas, comandos, variables, campos de base de datos u operaciones GraphQL.
Bloque gris con barra azul	Fragmento de código, comando de terminal o contenido de un archivo de configuración.
Implementado	La funcionalidad existe en el código y se puede usar desde la aplicación.
Parcial	Existe parte de la lógica (por ejemplo, la API) pero falta la interfaz o una regla completa.
Pendiente	Diseñada en la versión 1.0 del manual, pero aún no implementada en el código.
Rutas de archivo	Se expresan relativas a la raíz de cada repositorio, por ejemplo `src/asistente/asistente.service.ts`.

1.5 Glosario

Tabla 3. *Glosario de términos técnicos*

Término	Definición
Adulto mayor	Usuario principal del sistema (rol `Adulto Mayor`). Registra y consulta su rutina médica y conversa con Luma.
Cuidador	Usuario secundario (rol `Cuidador`), familiar o red de apoyo, que se vincula a un adulto mayor mediante un PIN.
Apollo Server	Servidor GraphQL que NestJS utiliza a través del controlador `@nestjs/apollo`.
Asistente Luma	Agente conversacional que responde preguntas sobre la agenda y las tomas del día, sin emitir diagnósticos.
bcrypt	Algoritmo de hash con sal usado para almacenar contraseñas de forma irreversible.
Code-first	Enfoque en el que el esquema GraphQL se genera a partir de clases TypeScript decoradas (`@ObjectType`, `@Field`).
DTO / Input	Objeto de transferencia de datos que define la forma de los argumentos de una mutación (`@InputType`).
EAS	Expo Application Services: servicio para compilar y publicar la app (.apk, .aab, .ipa).
Expo Go	Aplicación cliente que ejecuta el proyecto Expo en un dispositivo físico durante el desarrollo.
FastAPI	Framework web de Python con el que se construyó el motor de inferencia `LumaIA Engine API`.
GraphQL	Lenguaje de consulta para APIs. Se compone de *queries* (lectura) y *mutations* (escritura) sobre un único endpoint `/graphql`.
Índice TTL	Índice de MongoDB que elimina documentos automáticamente después de un tiempo. Implementa el borrado definitivo a 30 días.
Intención	Clasificación de lo que el usuario quiere hacer (por ejemplo `CONSULTA_GENERAL`, `ACCION_NUEVA_CITA`).
JWT	JSON Web Token. Credencial firmada que la app envía en el encabezado `Authorization: Bearer`.

Término	Definición
MongoDB Atlas	Servicio de MongoDB en la nube donde se aloja la base `lumaia` para el entorno compartido.
MongoDB Compass	Interfaz gráfica oficial de MongoDB que el equipo usa para administrar la base local: ver colecciones, editar documentos y ejecutar scripts con su consola mongosh integrada.
Mongoose	ODM (Object Document Mapper) que define esquemas y modelos sobre MongoDB.
ngrok	Servicio de túnel que publica el backend local (puerto 3000) en una URL HTTPS accesible desde el teléfono.
ObjectId	Identificador único de 12 bytes que MongoDB asigna a cada documento (`_id`).
Ollama / Llama 3.2	Ollama ejecuta modelos de lenguaje en el equipo local. Llama 3.2 es el modelo usado en el modo "Llama 3.2 Local".
PEAS	Marco para describir un agente inteligente: medida de desempeño, entorno, actuadores y sensores.
React Query	Librería (`@tanstack/react-query`) que gestiona peticiones, caché e invalidación de datos del servidor en la app.
RD-01	Regla de restricción diagnóstica: el asistente nunca debe diagnosticar ni modificar tratamientos.
Resolver	Clase de NestJS que expone *queries* y *mutations* GraphQL y delega la lógica en un *service*.
Soft delete	Borrado lógico: el documento se marca con `fecha_eliminacion` en lugar de eliminarse de inmediato.
Toma / Registro de toma	Evento en el que el adulto mayor confirma que ingirió un medicamento; se guarda en `registros_toma`.
TTS / STT	Text-to-Speech (texto a voz, `expo-speech`) y Speech-to-Text (voz a texto, Web Speech API en web).
Touch target	Área táctil de un botón. En LumaIA se amplía (48–64 px de alto) para usuarios con motricidad reducida.
Algoritmo genético	Metaheurística que evoluciona una población de soluciones mediante selección, cruce y mutación (ver 3.9).
BFS	Búsqueda en anchura: recorre un grafo por niveles usando una cola (ver 3.9).
DFS	Búsqueda en profundidad: recorre una rama hasta el fondo antes de retroceder, usando una pila (ver 3.9).
Distancia Euclidiana	Distancia en línea recta entre dos vectores: $\sqrt{\Sigma(x_i - y_i)^2}$ (ver 3.9).
Distancia Manhattan	Suma de diferencias absolutas entre coordenadas: $\Sigma\|x_i - y_i\|$ (ver 3.9).
Hill climbing	Búsqueda local que avanza al vecino que mejora la función objetivo (ver 3.9).
Tabla de verdad	Tabla que evalúa una expresión lógica para todas las combinaciones de valores de sus variables (ver 2.5).
UCS	Búsqueda de costo uniforme: expande el nodo de menor costo acumulado (ver 3.9).
WCAG	Pautas de accesibilidad para contenido web del W3C; se toma como referencia el nivel AA (contraste 4.5:1).
Zustand	Librería ligera de estado global usada para la sesión (`useAuthStore`) y el tema visual (`useThemeStore`).

2. Descripción general y requisitos funcionales y no funcionales

2.1 Descripción general del sistema

LumaIA atiende un problema concreto: los adultos mayores suelen tener tratamientos con varios medicamentos y citas médicas frecuentes, y las aplicaciones de uso general les resultan complejas. El sistema ofrece una interfaz de botones grandes y alto contraste con cuatro áreas principales: **Inicio** (el día de hoy), **Luma** (asistente conversacional), **Agenda** (medicamentos, citas y notas) y **Apoyo** (contactos de emergencia y botón SOS).

El asistente Luma es un agente basado en reglas y, de forma opcional, en un modelo de lenguaje local. Antes de responder consulta en la base de datos los medicamentos activos, las citas del día y las tomas ya confirmadas, de modo que sus respuestas se basan en el contexto real del usuario. Por diseño, el asistente no emite diagnósticos médicos (regla RD-01).

2.2 Actores del sistema

Tabla 4. *Actores*

Actor	Tipo	Interacción principal
Adulto mayor	Humano (usuario principal)	Registra medicamentos, citas y notas; confirma tomas; conversa con Luma; usa el botón SOS.
Cuidador	Humano (usuario secundario)	Se registra con rol Cuidador y se vincula al adulto mayor mediante PIN (API disponible).
Motor IA	Sistema externo	Recibe el contexto del día y genera la respuesta del asistente (FastAPI u Ollama).
MongoDB	Sistema externo	Persiste usuarios, rutina médica, tomas y bitácora de interacciones.

2.3 Marco PEAS del agente Luma

Tabla 5. *Descripción PEAS del agente inteligente*

Elemento	Descripción en LumaIA
P – Medida de desempeño	Respuestas correctas sobre tomas pendientes, tomas realizadas y citas del día; tiempo de respuesta menor a 2 s en modo Luma Cloud; cero diagnósticos médicos (RD-01); interfaz con contraste mínimo 4.5:1 y botones ampliados.
E – Entorno	Teléfono Android/iOS o navegador web; red con acceso al backend a través de ngrok; base de datos MongoDB con la rutina del usuario; reloj del servidor para determinar "hoy".
A – Actuadores	Texto en la burbuja de chat; voz sintetizada con `expo-speech` (es-MX, velocidad 0.9); registro en `logs_interaccion`; llamada telefónica del botón SOS (`tel:`).
S – Sensores	Botones rápidos y teclado; micrófono (dictado en web, grabación en móvil); consultas a las colecciones `medicamentos`, `citas` y `registros_toma`; token JWT de sesión.

2.4 Variables proposicionales

Para modelar el comportamiento del agente Luma y del sistema de alertas se definen seis variables proposicionales booleanas. Cada una toma el valor **V** (verdadero) o **F** (falso) y corresponde a un evento observable en el sistema.

Tabla 6. *Definición de variables proposicionales*

Variable	Proposición	Cómo se observa en el sistema
P	El recordatorio de un medicamento crítico ha sido emitido al usuario.	Llega la hora de un elemento de `horarios_especificos` del medicamento.
Q	El adulto mayor confirmó la toma dentro del límite de 15 minutos.	Existe un documento en `registros_toma` con `estado_toma = CONFIRMADA` dentro del plazo.
R	El sistema envía una alerta al cuidador o a la red de apoyo.	Notificación al cuidador o llamada a un contacto de emergencia.
S	El usuario presiona el botón SOS.	Pulsación de **EMERGENCIA SOS** / **LLAMADA DE EMERGENCIA**.
D	La consulta ingresada contiene una solicitud de diagnóstico clínico o prescripción médica.	`pregunta_id` con intención de síntoma, dolor, diagnóstico o ajuste de dosis.
B	El motor lógico activa el bloqueo médico y restringe la respuesta (regla RD-01).	Respuesta de negativa y `bloqueo_medico = true` en `logs_interaccion`.

Con estas variables se formulan las tres reglas de negocio que el agente debe cumplir:

Tabla 7. *Reglas de negocio en lógica proposicional*

Regla	Nombre	Expresión	Lectura
R1 (RN-07)	Escalamiento por omisión	$(P \wedge \neg Q) \rightarrow R$	Si se emitió el recordatorio y no se confirmó la toma, se alerta al cuidador.
R2 (RD-01)	Restricción diagnóstica	$D \rightarrow B$	Si la consulta pide un diagnóstico, se activa el bloqueo médico.
R3 (RN-08)	Activación del botón SOS	$S \rightarrow R$	Si se presiona el botón SOS, se dispara la alerta a la red de apoyo.

El sistema se considera **en un estado válido** cuando se cumplen las tres reglas al mismo tiempo: **R1 ∧ R2 ∧ R3**.

2.5 Tablas de verdad

Las cuatro tablas evalúan las **64 combinaciones posibles** (2^6) de las seis variables, ordenadas de VVVVVV a FFFFFF. Las filas sombreadas en rojo son los casos en los que la regla evaluada resulta **falsa**, es decir, situaciones que el sistema nunca debe permitir. Estas tablas sirven además como oráculo para las pruebas de la sección 9.

Tabla de verdad general

Evalúa las tres reglas y su conjunción. El sistema es válido en **33 de 64** combinaciones; en las 31 restantes se incumple al menos una regla.

Tabla 8. *Tabla de verdad general: R1 ∧ R2 ∧ R3*

#	P	Q	R	S	D	B	¬Q	P ∧ ¬Q	R1: (P∧¬Q)→R	R2: D→B	R3: S→R	R1 ∧ R2 ∧ R3
1	V	V	V	V	V	V	F	F	V	V	V	**V**
2	V	V	V	V	V	F	F	F	V	F	V	**F**
3	V	V	V	V	F	V	F	F	V	V	V	**V**
4	V	V	V	V	F	F	F	F	V	V	V	**V**
5	V	V	V	F	V	V	F	F	V	V	V	**V**
6	V	V	V	F	V	F	F	F	V	F	V	**F**
7	V	V	V	F	F	V	F	F	V	V	V	**V**
8	V	V	V	F	F	F	F	F	V	V	V	**V**
9	V	V	F	V	V	V	F	F	V	V	F	**F**
10	V	V	F	V	V	F	F	F	V	F	F	**F**
11	V	V	F	V	F	V	F	F	V	V	F	**F**
12	V	V	F	V	F	F	F	F	V	V	F	**F**
13	V	V	F	F	V	V	F	F	V	V	V	**V**
14	V	V	F	F	V	F	F	F	V	F	V	**F**
15	V	V	F	F	F	V	F	F	V	V	V	**V**
16	V	V	F	F	F	F	F	F	V	V	V	**V**
17	V	F	V	V	V	V	V	V	V	V	V	**V**
18	V	F	V	V	V	F	V	V	V	F	V	**F**
19	V	F	V	V	F	V	V	V	V	V	V	**V**
20	V	F	V	V	F	F	V	V	V	V	V	**V**
21	V	F	V	F	V	V	V	V	V	V	V	**V**
22	V	F	V	F	V	F	V	V	V	F	V	**F**
23	V	F	V	F	F	V	V	V	V	V	V	**V**
24	V	F	V	F	F	F	V	V	V	V	V	**V**
25	V	F	F	V	V	V	V	V	F	V	F	**F**
26	V	F	F	V	V	F	V	V	F	F	F	**F**
27	V	F	F	V	F	V	V	V	F	V	F	**F**
28	V	F	F	V	F	F	V	V	F	V	F	**F**
29	V	F	F	F	V	V	V	V	F	V	V	**F**
30	V	F	F	F	V	F	V	V	F	F	V	**F**
31	V	F	F	F	F	V	V	V	F	V	V	**F**
32	V	F	F	F	F	F	V	V	F	V	V	**F**
33	F	V	V	V	V	V	F	F	V	V	V	**V**
34	F	V	V	V	V	F	F	F	V	F	V	**F**
35	F	V	V	V	F	V	F	F	V	V	V	**V**
36	F	V	V	V	F	F	F	F	V	V	V	**V**
37	F	V	V	F	V	V	F	F	V	V	V	**V**

#	P	Q	R	S	D	B	¬Q	P ∧ ¬Q	R1: (P∧¬Q)→R	R2: D→B	R3: S→R	R1 ∧ R2 ∧ R3
38	F	V	V	F	V	F	F	F	V	F	V	**F**
39	F	V	V	F	F	V	F	F	V	V	V	**V**
40	F	V	V	F	F	F	F	F	V	V	V	**V**
41	F	V	F	V	V	V	F	F	V	V	F	**F**
42	F	V	F	V	V	F	F	F	V	F	F	**F**
43	F	V	F	V	F	V	F	F	V	V	F	**F**
44	F	V	F	V	F	F	F	F	V	V	F	**F**
45	F	V	F	F	V	V	F	F	V	V	V	**V**
46	F	V	F	F	V	F	F	F	V	F	V	**F**
47	F	V	F	F	F	V	F	F	V	V	V	**V**
48	F	V	F	F	F	F	F	F	V	V	V	**V**
49	F	F	V	V	V	V	V	F	V	V	V	**V**
50	F	F	V	V	V	F	V	F	V	F	V	**F**
51	F	F	V	V	F	V	V	F	V	V	V	**V**
52	F	F	V	V	F	F	V	F	V	V	V	**V**
53	F	F	V	F	V	V	V	F	V	V	V	**V**
54	F	F	V	F	V	F	V	F	V	F	V	**F**
55	F	F	V	F	F	V	V	F	V	V	V	**V**
56	F	F	V	F	F	F	V	F	V	V	V	**V**
57	F	F	F	V	V	V	V	F	V	V	F	**F**
58	F	F	F	V	V	F	V	F	V	F	F	**F**
59	F	F	F	V	F	V	V	F	V	V	F	**F**
60	F	F	F	V	F	F	V	F	V	V	F	**F**
61	F	F	F	F	V	V	V	F	V	V	V	**V**
62	F	F	F	F	V	F	V	F	V	F	V	**F**
63	F	F	F	F	F	V	V	F	V	V	V	**V**
64	F	F	F	F	F	F	V	F	V	V	V	**V**

Tabla de verdad de la regla R1: escalamiento por omisión

(P ∧ ¬Q) → R es falsa en **8 de 64** combinaciones: cuando se emitió el recordatorio (P = V), no se confirmó la toma (Q = F) y no se envió alerta (R = F). S, D y B no influyen en el resultado.

Tabla 9. *Tabla de verdad de R1: (P ∧ ¬Q) → R*

#	P	Q	R	S	D	B	¬Q	P ∧ ¬Q	(P∧¬Q)→R	Interpretación
1	V	V	V	V	V	V	F	F	**V**	Toma confirmada a tiempo
2	V	V	V	V	V	F	F	F	**V**	Toma confirmada a tiempo
3	V	V	V	V	F	V	F	F	**V**	Toma confirmada a tiempo
4	V	V	V	V	F	F	F	F	**V**	Toma confirmada a tiempo

#	P	Q	R	S	D	B	¬Q	P ∧ ¬Q	(P∧¬Q)→R	Interpretación
5	V	V	V	F	V	V	F	F	**V**	Toma confirmada a tiempo
6	V	V	V	F	V	F	F	F	**V**	Toma confirmada a tiempo
7	V	V	V	F	F	V	F	F	**V**	Toma confirmada a tiempo
8	V	V	V	F	F	F	F	F	**V**	Toma confirmada a tiempo
9	V	V	F	V	V	V	F	F	**V**	Toma confirmada a tiempo
10	V	V	F	V	V	F	F	F	**V**	Toma confirmada a tiempo
11	V	V	F	V	F	V	F	F	**V**	Toma confirmada a tiempo
12	V	V	F	V	F	F	F	F	**V**	Toma confirmada a tiempo
13	V	V	F	F	V	V	F	F	**V**	Toma confirmada a tiempo
14	V	V	F	F	V	F	F	F	**V**	Toma confirmada a tiempo
15	V	V	F	F	F	V	F	F	**V**	Toma confirmada a tiempo
16	V	V	F	F	F	F	F	F	**V**	Toma confirmada a tiempo
17	V	F	V	V	V	V	V	V	**V**	Omisión detectada, alerta enviada
18	V	F	V	V	V	F	V	V	**V**	Omisión detectada, alerta enviada
19	V	F	V	V	F	V	V	V	**V**	Omisión detectada, alerta enviada
20	V	F	V	V	F	F	V	V	**V**	Omisión detectada, alerta enviada
21	V	F	V	F	V	V	V	V	**V**	Omisión detectada, alerta enviada
22	V	F	V	F	V	F	V	V	**V**	Omisión detectada, alerta enviada
23	V	F	V	F	F	V	V	V	**V**	Omisión detectada, alerta enviada
24	V	F	V	F	F	F	V	V	**V**	Omisión detectada, alerta enviada
25	V	F	F	V	V	V	V	V	**F**	Fallo: omisión sin alerta
26	V	F	F	V	V	F	V	V	**F**	Fallo: omisión sin alerta
27	V	F	F	V	F	V	V	V	**F**	Fallo: omisión sin alerta
28	V	F	F	V	F	F	V	V	**F**	Fallo: omisión sin alerta
29	V	F	F	F	V	V	V	V	**F**	Fallo: omisión sin alerta
30	V	F	F	F	V	F	V	V	**F**	Fallo: omisión sin alerta
31	V	F	F	F	F	V	V	V	**F**	Fallo: omisión sin alerta
32	V	F	F	F	F	F	V	V	**F**	Fallo: omisión sin alerta
33	F	V	V	V	V	V	F	F	**V**	Sin recordatorio activo
34	F	V	V	V	V	F	F	F	**V**	Sin recordatorio activo
35	F	V	V	V	F	V	F	F	**V**	Sin recordatorio activo
36	F	V	V	V	F	F	F	F	**V**	Sin recordatorio activo
37	F	V	V	F	V	V	F	F	**V**	Sin recordatorio activo
38	F	V	V	F	V	F	F	F	**V**	Sin recordatorio activo
39	F	V	V	F	F	V	F	F	**V**	Sin recordatorio activo
40	F	V	V	F	F	F	F	F	**V**	Sin recordatorio activo
41	F	V	F	V	V	V	F	F	**V**	Sin recordatorio activo
42	F	V	F	V	V	F	F	F	**V**	Sin recordatorio activo

#	P	Q	R	S	D	B	¬Q	P ∧ ¬Q	(P∧¬Q)→R	Interpretación
43	F	V	F	V	F	V	F	F	**V**	Sin recordatorio activo
44	F	V	F	V	F	F	F	F	**V**	Sin recordatorio activo
45	F	V	F	F	V	V	F	F	**V**	Sin recordatorio activo
46	F	V	F	F	V	F	F	F	**V**	Sin recordatorio activo
47	F	V	F	F	F	V	F	F	**V**	Sin recordatorio activo
48	F	V	F	F	F	F	F	F	**V**	Sin recordatorio activo
49	F	F	V	V	V	V	V	F	**V**	Sin recordatorio activo
50	F	F	V	V	V	F	V	F	**V**	Sin recordatorio activo
51	F	F	V	V	F	V	V	F	**V**	Sin recordatorio activo
52	F	F	V	V	F	F	V	F	**V**	Sin recordatorio activo
53	F	F	V	F	V	V	V	F	**V**	Sin recordatorio activo
54	F	F	V	F	V	F	V	F	**V**	Sin recordatorio activo
55	F	F	V	F	F	V	V	F	**V**	Sin recordatorio activo
56	F	F	V	F	F	F	V	F	**V**	Sin recordatorio activo
57	F	F	F	V	V	V	V	F	**V**	Sin recordatorio activo
58	F	F	F	V	V	F	V	F	**V**	Sin recordatorio activo
59	F	F	F	V	F	V	V	F	**V**	Sin recordatorio activo
60	F	F	F	V	F	F	V	F	**V**	Sin recordatorio activo
61	F	F	F	F	V	V	V	F	**V**	Sin recordatorio activo
62	F	F	F	F	V	F	V	F	**V**	Sin recordatorio activo
63	F	F	F	F	F	V	V	F	**V**	Sin recordatorio activo
64	F	F	F	F	F	F	V	F	**V**	Sin recordatorio activo

Tabla de verdad de la regla R2: restricción diagnóstica

D → B es falsa en **16 de 64** combinaciones: cuando la consulta solicita un diagnóstico (D = V) y el bloqueo no se activa (B = F). Es el fallo más grave, porque permitiría que el asistente diagnosticara.

Tabla 10. *Tabla de verdad de R2: D → B*

#	P	Q	R	S	D	B	D → B	Interpretación
1	V	V	V	V	V	V	**V**	Bloqueo correcto
2	V	V	V	V	V	F	**F**	Fallo crítico: diagnóstico permitido
3	V	V	V	V	F	V	**V**	Bloqueo defensivo
4	V	V	V	V	F	F	**V**	Consulta normal de agenda
5	V	V	V	F	V	V	**V**	Bloqueo correcto
6	V	V	V	F	V	F	**F**	Fallo crítico: diagnóstico permitido
7	V	V	V	F	F	V	**V**	Bloqueo defensivo
8	V	V	V	F	F	F	**V**	Consulta normal de agenda
9	V	V	F	V	V	V	**V**	Bloqueo correcto

#	P	Q	R	S	D	B	D → B	Interpretación
10	V	V	F	V	V	F	**F**	Fallo crítico: diagnóstico permitido
11	V	V	F	V	F	V	**V**	Bloqueo defensivo
12	V	V	F	V	F	F	**V**	Consulta normal de agenda
13	V	V	F	F	V	V	**V**	Bloqueo correcto
14	V	V	F	F	V	F	**F**	Fallo crítico: diagnóstico permitido
15	V	V	F	F	F	V	**V**	Bloqueo defensivo
16	V	V	F	F	F	F	**V**	Consulta normal de agenda
17	V	F	V	V	V	V	**V**	Bloqueo correcto
18	V	F	V	V	V	F	**F**	Fallo crítico: diagnóstico permitido
19	V	F	V	V	F	V	**V**	Bloqueo defensivo
20	V	F	V	V	F	F	**V**	Consulta normal de agenda
21	V	F	V	F	V	V	**V**	Bloqueo correcto
22	V	F	V	F	V	F	**F**	Fallo crítico: diagnóstico permitido
23	V	F	V	F	F	V	**V**	Bloqueo defensivo
24	V	F	V	F	F	F	**V**	Consulta normal de agenda
25	V	F	F	V	V	V	**V**	Bloqueo correcto
26	V	F	F	V	V	F	**F**	Fallo crítico: diagnóstico permitido
27	V	F	F	V	F	V	**V**	Bloqueo defensivo
28	V	F	F	V	F	F	**V**	Consulta normal de agenda
29	V	F	F	F	V	V	**V**	Bloqueo correcto
30	V	F	F	F	V	F	**F**	Fallo crítico: diagnóstico permitido
31	V	F	F	F	F	V	**V**	Bloqueo defensivo
32	V	F	F	F	F	F	**V**	Consulta normal de agenda
33	F	V	V	V	V	V	**V**	Bloqueo correcto
34	F	V	V	V	V	F	**F**	Fallo crítico: diagnóstico permitido
35	F	V	V	V	F	V	**V**	Bloqueo defensivo
36	F	V	V	V	F	F	**V**	Consulta normal de agenda
37	F	V	V	F	V	V	**V**	Bloqueo correcto
38	F	V	V	F	V	F	**F**	Fallo crítico: diagnóstico permitido
39	F	V	V	F	F	V	**V**	Bloqueo defensivo
40	F	V	V	F	F	F	**V**	Consulta normal de agenda
41	F	V	F	V	V	V	**V**	Bloqueo correcto
42	F	V	F	V	V	F	**F**	Fallo crítico: diagnóstico permitido
43	F	V	F	V	F	V	**V**	Bloqueo defensivo
44	F	V	F	V	F	F	**V**	Consulta normal de agenda
45	F	V	F	F	V	V	**V**	Bloqueo correcto
46	F	V	F	F	V	F	**F**	Fallo crítico: diagnóstico permitido
47	F	V	F	F	F	V	**V**	Bloqueo defensivo

#	P	Q	R	S	D	B	D → B	Interpretación
48	F	V	F	F	F	F	**V**	Consulta normal de agenda
49	F	F	V	V	V	V	**V**	Bloqueo correcto
50	F	F	V	V	V	F	**F**	Fallo crítico: diagnóstico permitido
51	F	F	V	V	F	V	**V**	Bloqueo defensivo
52	F	F	V	V	F	F	**V**	Consulta normal de agenda
53	F	F	V	F	V	V	**V**	Bloqueo correcto
54	F	F	V	F	V	F	**F**	Fallo crítico: diagnóstico permitido
55	F	F	V	F	F	V	**V**	Bloqueo defensivo
56	F	F	V	F	F	F	**V**	Consulta normal de agenda
57	F	F	F	V	V	V	**V**	Bloqueo correcto
58	F	F	F	V	V	F	**F**	Fallo crítico: diagnóstico permitido
59	F	F	F	V	F	V	**V**	Bloqueo defensivo
60	F	F	F	V	F	F	**V**	Consulta normal de agenda
61	F	F	F	F	V	V	**V**	Bloqueo correcto
62	F	F	F	F	V	F	**F**	Fallo crítico: diagnóstico permitido
63	F	F	F	F	F	V	**V**	Bloqueo defensivo
64	F	F	F	F	F	F	**V**	Consulta normal de agenda

Tabla de verdad de la regla R3: activación del botón SOS

S → R es falsa en **16 de 64** combinaciones: cuando se presiona el botón SOS (S = V) y no se dispara la alerta (R = F). Note que R también puede ser verdadera por la regla R1 aunque no se presione el botón.

Tabla 11. *Tabla de verdad de R3: S → R*

#	P	Q	R	S	D	B	S → R	Interpretación
1	V	V	V	V	V	V	**V**	Emergencia notificada
2	V	V	V	V	V	F	**V**	Emergencia notificada
3	V	V	V	V	F	V	**V**	Emergencia notificada
4	V	V	V	V	F	F	**V**	Emergencia notificada
5	V	V	V	F	V	V	**V**	Alerta por otra causa (R1)
6	V	V	V	F	V	F	**V**	Alerta por otra causa (R1)
7	V	V	V	F	F	V	**V**	Alerta por otra causa (R1)
8	V	V	V	F	F	F	**V**	Alerta por otra causa (R1)
9	V	V	F	V	V	V	**F**	Fallo: botón SOS sin alerta
10	V	V	F	V	V	F	**F**	Fallo: botón SOS sin alerta
11	V	V	F	V	F	V	**F**	Fallo: botón SOS sin alerta
12	V	V	F	V	F	F	**F**	Fallo: botón SOS sin alerta
13	V	V	F	F	V	V	**V**	Reposo habitual
14	V	V	F	F	V	F	**V**	Reposo habitual

#	P	Q	R	S	D	B	S → R	Interpretación
15	V	V	F	F	F	V	**V**	Reposo habitual
16	V	V	F	F	F	F	**V**	Reposo habitual
17	V	F	V	V	V	V	**V**	Emergencia notificada
18	V	F	V	V	V	F	**V**	Emergencia notificada
19	V	F	V	V	F	V	**V**	Emergencia notificada
20	V	F	V	V	F	F	**V**	Emergencia notificada
21	V	F	V	F	V	V	**V**	Alerta por otra causa (R1)
22	V	F	V	F	V	F	**V**	Alerta por otra causa (R1)
23	V	F	V	F	F	V	**V**	Alerta por otra causa (R1)
24	V	F	V	F	F	F	**V**	Alerta por otra causa (R1)
25	V	F	F	V	V	V	**F**	Fallo: botón SOS sin alerta
26	V	F	F	V	V	F	**F**	Fallo: botón SOS sin alerta
27	V	F	F	V	F	V	**F**	Fallo: botón SOS sin alerta
28	V	F	F	V	F	F	**F**	Fallo: botón SOS sin alerta
29	V	F	F	F	V	V	**V**	Reposo habitual
30	V	F	F	F	V	F	**V**	Reposo habitual
31	V	F	F	F	F	V	**V**	Reposo habitual
32	V	F	F	F	F	F	**V**	Reposo habitual
33	F	V	V	V	V	V	**V**	Emergencia notificada
34	F	V	V	V	V	F	**V**	Emergencia notificada
35	F	V	V	V	F	V	**V**	Emergencia notificada
36	F	V	V	V	F	F	**V**	Emergencia notificada
37	F	V	V	F	V	V	**V**	Alerta por otra causa (R1)
38	F	V	V	F	V	F	**V**	Alerta por otra causa (R1)
39	F	V	V	F	F	V	**V**	Alerta por otra causa (R1)
40	F	V	V	F	F	F	**V**	Alerta por otra causa (R1)
41	F	V	F	V	V	V	**F**	Fallo: botón SOS sin alerta
42	F	V	F	V	V	F	**F**	Fallo: botón SOS sin alerta
43	F	V	F	V	F	V	**F**	Fallo: botón SOS sin alerta
44	F	V	F	V	F	F	**F**	Fallo: botón SOS sin alerta
45	F	V	F	F	V	V	**V**	Reposo habitual
46	F	V	F	F	V	F	**V**	Reposo habitual
47	F	V	F	F	F	V	**V**	Reposo habitual
48	F	V	F	F	F	F	**V**	Reposo habitual
49	F	F	V	V	V	V	**V**	Emergencia notificada
50	F	F	V	V	V	F	**V**	Emergencia notificada
51	F	F	V	V	F	V	**V**	Emergencia notificada
52	F	F	V	V	F	F	**V**	Emergencia notificada

#	P	Q	R	S	D	B	S → R	Interpretación
53	F	F	V	F	V	V	**V**	Alerta por otra causa (R1)
54	F	F	V	F	V	F	**V**	Alerta por otra causa (R1)
55	F	F	V	F	F	V	**V**	Alerta por otra causa (R1)
56	F	F	V	F	F	F	**V**	Alerta por otra causa (R1)
57	F	F	F	V	V	V	**F**	Fallo: botón SOS sin alerta
58	F	F	F	V	V	F	**F**	Fallo: botón SOS sin alerta
59	F	F	F	V	F	V	**F**	Fallo: botón SOS sin alerta
60	F	F	F	V	F	F	**F**	Fallo: botón SOS sin alerta
61	F	F	F	F	V	V	**V**	Reposo habitual
62	F	F	F	F	V	F	**V**	Reposo habitual
63	F	F	F	F	F	V	**V**	Reposo habitual
64	F	F	F	F	F	F	**V**	Reposo habitual

Estado en el código: R2 se cumple solo para códigos con "SINTOMA" o "DOLOR" (ver sección 11); R3 se cumple mediante la llamada telefónica al primer contacto; R1 está pendiente porque aún no existen recordatorios ni notificaciones push.

2.6 Requisitos funcionales

La siguiente tabla relaciona cada requisito funcional con su estado real en el código de la versión 2.0 y con el archivo donde se implementa. Esta trazabilidad es la referencia principal para el mantenimiento.

Tabla 12. *Requisitos funcionales y su trazabilidad al código*

ID	Módulo	Requisito	Estado	Evidencia en código
RF-01	Usuarios	Registrar cuenta con nombre, correo, contraseña (mín. 6 caracteres) y rol.	Implementado	`registrarUsuario` · RegisterScreen
RF-02	Usuarios	Iniciar sesión con correo y contraseña.	Implementado	`login` · LoginScreen
RF-03	Usuarios	Mantener la sesión abierta (JWT con vigencia de 30 días guardado en el dispositivo).	Implementado	`auth.module.ts` · `useAuthStore`
RF-04	Usuarios	Vincular cuidador y adulto mayor mediante PIN de 6 dígitos de un solo uso.	Parcial	API `generarPinVinculacion`, `vincularCuidador`; sin pantalla
RF-05	Usuarios	Agregar y consultar contactos de emergencia (red de apoyo).	Implementado	`agregarContactoEmergencia` · ApoyoScreen
RF-06	Rutina	Registrar medicamento con dosis, frecuencia y hora de primera toma; calcular los horarios del día.	Implementado	`MedicamentosService.create` · AgendaScreen
RF-07	Rutina	Consultar medicamentos activos del usuario.	Implementado	`obtenerMedicamentosPorUsuario`
RF-08	Rutina	Confirmar una toma ("Ya lo tomé") y consultar las tomas confirmadas de hoy.	Implementado	`confirmarTomaMedicamento`, `obtenerTomasHoy` · DashboardScreen

ID	Módulo	Requisito	Estado	Evidencia en código
RF-09	Rutina	Programar y consultar citas médicas con fecha, hora y ubicación.	Implementado	`crearCita`, `obtenerCitasPorUsuario`
RF-10	Rutina	Crear y consultar notas personales.	Implementado	`crearNotaPersonal`, `obtenerNotasPersonalesPorUsuario`
RF-11	Rutina	Borrado lógico con eliminación definitiva automática a los 30 días.	Parcial	Campo `fecha_eliminacion` + índice TTL; no existe mutación de borrado
RF-12	Interfaz	Mostrar el panel del día con medicamentos y citas de hoy.	Implementado	DashboardScreen
RF-13	Interfaz	Mostrar la agenda filtrable por Todo, Citas, Medicamentos y Notas.	Implementado	AgendaScreen
RF-14	Interfaz	Ofrecer modo de alto contraste y tamaño de letra ampliado (x1.2 y x1.4).	Parcial	`useThemeStore`; falta pantalla de ajustes
RF-15	Asistente	Responder consultas por botones rápidos o texto libre y leer la respuesta en voz alta.	Implementado	`consultarAsistente` · ChatScreen · `expo-speech`
RF-16	Asistente	Elegir el motor de IA: Luma Cloud (FastAPI) o Llama 3.2 Local (Ollama).	Implementado	`AsistenteService.procesarPregunta`
RF-17	Asistente	Dictar la pregunta por voz (Speech-to-Text).	Parcial	Web: Web Speech API. Móvil: graba audio; transcripción pendiente
RF-18	Seguridad	Bloquear solicitudes de diagnóstico o ajuste de dosis (RD-01).	Parcial	`main.py` (códigos SINTOMA/DOLOR); ver sección 11
RF-19	Seguridad	Botón SOS que inicia llamada al primer contacto de emergencia.	Implementado	ApoyoScreen `handleSOS`
RF-20	Seguridad	Notificar al cuidador si no se confirma una toma crítica en 15 min.	Pendiente	Solo existe el campo `tokens_dispositivo`
RF-21	Seguridad	Registrar en bitácora cada interacción con el asistente.	Implementado	Colección `logs_interaccion`

2.7 Requisitos no funcionales

Tabla 13. *Requisitos no funcionales*

ID	Atributo	Requisito	Cómo se atiende en el código
RNF-01	Usabilidad y accesibilidad	Interfaz legible para adultos mayores, con áreas táctiles amplias y contraste alto.	Texto base de 16 pt escalable hasta 32 pt; botones de 48 a 64 px de alto; paleta de alto contraste en `useThemeStore`; navegación de un solo nivel con 4 pestañas.
RNF-02	Rendimiento	Respuesta del asistente menor a 2 s en modo Luma Cloud.	Motor de reglas en FastAPI sin llamadas externas. Para Ollama el cliente tolera hasta 45 s (`timeout: 45000`).
RNF-03	Disponibilidad	La sesión debe sobrevivir al cierre de la app.	Sesión persistida con Zustand + AsyncStorage. Los datos se guardan en caché en memoria con React Query (sin modo offline completo).
RNF-04	Portabilidad	Funcionar en Android, iOS y navegador web.	Expo con `react-native-web`; scripts `android`, `ios` y `web`.
RNF-05	Mantenibilidad	Código modular y homogéneo.	Un módulo NestJS por dominio; TypeScript; ESLint + Prettier en el backend; `tsconfig` estricto en la app.
RNF-06	Seguridad	Contraseñas nunca almacenadas en texto plano.	Hash bcrypt con sal de 10 rondas (`usuarios.service.ts`).
RNF-07	Ética y privacidad	No emitir diagnósticos; permitir procesamiento privado.	Respuestas de bloqueo en el motor IA; opción de modelo local Llama 3.2 vía Ollama.

2.8 Reglas de negocio

Las reglas de negocio se expresan en lógica proposicional, coherente con el enfoque de la asignatura. Las reglas R1, R2 y R3 se analizan con sus tablas de verdad en la sección 2.5.

Tabla 14. *Reglas de negocio implementadas y pendientes*

ID	Regla	Expresión / parámetro	Estado
RD-01	Restricción diagnóstica: si la consulta pide diagnóstico (D), el asistente bloquea la respuesta (B).	$D \rightarrow B$	Parcial
RD-02	Retención: un registro marcado como eliminado se purga a los 30 días.	`expireAfterSeconds: 2592000`	Parcial
RN-03	Horarios: a partir de la primera toma y la frecuencia se generan ⌊24 / frecuencia⌋ horarios, usando módulo 24 para cruzar la medianoche.	$h_i = (h_0 + i \cdot f) \bmod 24$	Implementado
RN-04	Una toma confirmada por medicamento por día.	Validado en la interfaz	Parcial
RN-05	El correo electrónico es único por cuenta.	Índice único `correo`	Implementado
RN-06	El PIN de vinculación se elimina al usarse.	`$unset: pin_vinculacion`	Implementado
RN-07	Escalamiento: recordatorio emitido (P) sin confirmación (¬Q) notifica al cuidador (R).	$(P \wedge \neg Q) \rightarrow R$	Pendiente
RN-08	Botón SOS: al presionarlo (S) se dispara la alerta a la red de apoyo (R).	$S \rightarrow R$	Parcial (llamada)
RN-09	Una cuenta con `estado_activo = false` no puede iniciar sesión.	Error `FORBIDDEN`	Implementado

2.9 Historias de usuario

Tabla 15. *Historias de usuario y requisitos asociados*

ID	Historia de usuario	RF
HU-01	Como usuario, quiero registrarme y elegir mi rol (adulto mayor o cuidador) para ver una interfaz adaptada a mí.	RF-01, RF-02, RF-03
HU-02	Como cuidador, quiero vincularme con un adulto mayor mediante un PIN para apoyar su agenda médica.	RF-04
HU-03	Como adulto mayor, quiero registrar mis medicamentos con su dosis y horario para no olvidar mis tratamientos.	RF-06, RF-07, RF-08
HU-04	Como adulto mayor, quiero preguntarle a Luma "¿cuál es mi próxima medicina?" y escuchar la respuesta.	RF-15, RF-16, RF-17
HU-05	Como adulto mayor, quiero un botón SOS siempre visible para llamar a mi red de apoyo.	RF-05, RF-19
HU-06	Como cuidador, quiero recibir una alerta si no se confirma una toma crítica en 15 minutos.	RF-20

3. Arquitectura y tecnologías

3.1 Estilo arquitectónico

LumaIA sigue una arquitectura **cliente-servidor de tres capas** con un **servicio de inteligencia desacoplado**:

- **Presentación:** aplicación Expo/React Native. Las pantallas (`screens`) llaman a servicios (`services`) que envían operaciones GraphQL mediante un único cliente Axios (`api.client.ts`). React Query administra el estado del servidor y Zustand el estado local (sesión y tema).
- **Negocio:** API NestJS organizada por módulos de dominio. Cada módulo sigue el patrón *Resolver → Service → Model*: el resolver expone la operación GraphQL, el service contiene la lógica y el modelo Mongoose accede a la colección.
- **Datos:** MongoDB con la base `lumaia` y seis colecciones. En desarrollo se usa un servidor MongoDB local administrado con **MongoDB Compass**; en la nube se usa **MongoDB Atlas**. Ambos comparten el mismo esquema y solo cambia la cadena `MONGODB_URI`.
- **Inteligencia:** el `AsistenteService` arma el contexto y decide a qué motor enviar la consulta: el servicio FastAPI (modo *Luma Cloud*) o Ollama con Llama 3.2 (modo *Llama 3.2 Local*).

La API es **GraphQL de un solo endpoint** (`POST /graphql`) construida con enfoque *code-first*: el archivo `src/schema.gql` se regenera automáticamente cada vez que arranca el backend.

3.2 Componentes y comunicación

Tabla 16. *Componentes del sistema y su comunicación*

Origen	Destino	Protocolo / medio	Propósito
App móvil (Expo)	Túnel ngrok o Nginx	HTTPS, `POST /graphql`, encabezado `Authorization: Bearer`	Enviar consultas y mutaciones GraphQL
Túnel ngrok o Nginx	API NestJS (puerto 3000)	HTTP local	Publicar el backend de forma segura
API NestJS	MongoDB local (Compass) o Atlas	Mongoose ODM, puerto 27017 o `mongodb+srv`	Leer y escribir las seis colecciones
API NestJS	Motor IA FastAPI (puerto 8000)	HTTP JSON, `POST /evaluar`	Respuestas del modo Luma Cloud
API NestJS	Ollama (puerto 11434)	HTTP JSON, `POST /api/generate`	Respuestas del modo Llama 3.2 Local

3.3 Flujo de una consulta al asistente

1. El adulto mayor pulsa un botón rápido o escribe su pregunta en la pantalla Luma.
2. La app envía la mutación `consultarAsistente(usuario_id, pregunta_id, modelo)`.

3. El `AsistenteService` consulta en MongoDB los medicamentos activos, las citas de hoy y las tomas confirmadas de hoy.
4. Cruza la información para separar medicamentos pendientes y tomados, y arma el *system prompt*.
5. Envía la consulta al motor elegido: `POST /evaluar` (FastAPI) o `POST /api/generate` (Ollama).
6. Recibe la respuesta, la normaliza y la registra en `logs_interaccion`.
7. Devuelve `respuesta_texto` e `intencion`; la app muestra la burbuja y la lee en voz alta con `expo-speech`.

El backend no confía en que el motor de IA conozca la base de datos: es el `AsistenteService` quien consulta MongoDB, separa los medicamentos **pendientes** de los **ya tomados** comparando sus identificadores como cadenas y envía al motor solo el contexto necesario. Al final guarda la pregunta y la respuesta en `logs_interaccion` para auditoría.

3.4 Pila tecnológica

Tabla 17. *Tecnologías y versiones declaradas en los proyectos*

Capa	Tecnología	Versión	Uso en LumaIA
App móvil	Expo SDK	~57.0.30	Plataforma de desarrollo y compilación
App móvil	React / React Native	19.2.3 / 0.86.3	Interfaz nativa multiplataforma
App móvil	TypeScript	~6.0.3	Tipado estático (modo `strict`)
App móvil	React Navigation	^7.x	Pila de autenticación y pestañas inferiores
App móvil	@tanstack/react-query	^5.104.4	Peticiones, caché e invalidación
App móvil	Zustand + AsyncStorage	^5.0.15 / 2.2.0	Sesión persistente y tema visual
App móvil	Axios	^1.20.0	Cliente HTTP hacia `/graphql`
App móvil	expo-speech / expo-av / expo-file-system	~57.0.3 / ^16.0.8 / ~57.0.7	Voz sintetizada, grabación de audio y lectura en Base64
App móvil	@react-native-community/datetimepicker	9.1.0	Selección de fecha y hora
App móvil	lucide-react-native	^1.59.0	Iconografía
Backend	NestJS (core, common, platform-express)	^11.0.1	Framework del servidor
Backend	@nestjs/graphql + @nestjs/apollo + @apollo/server	^14.0.3 / ^5.5.1	Servidor GraphQL code-first
Backend	graphql	^17.0.2	Implementación de referencia de GraphQL
Backend	@nestjs/mongoose + mongoose	^12.0.0 / ^9.10.3	Acceso a MongoDB
Backend	@nestjs/jwt + bcrypt	^12.0.2 / ^6.0.0	Tokens de sesión y hash de contraseñas
Backend	class-validator + class-transformer	^0.15.1 / ^0.5.1	Validación de DTOs
Backend	@nestjs/axios + @nestjs/config	^12.0.1	Llamadas HTTP al motor IA y variables de entorno
Backend	Jest + Supertest + ts-jest	^30.0.0 / ^7.0.0 / ^29.2.5	Pruebas unitarias y e2e

Capa	Tecnología	Versión	Uso en LumaIA
Motor IA	Python	3.10 o superior	Requerido por la sentencia `match` de `evaluator.py`
Motor IA	FastAPI + Pydantic + Uvicorn	Versión estable vigente	Endpoint `POST /evaluar`
Motor IA	Ollama + modelo `llama3.2`	Versión estable vigente	Modelo de lenguaje local
Datos	MongoDB Community Server (local)	7.x o superior recomendado	Base de datos local de desarrollo
Datos	MongoDB Compass	Versión estable vigente	Interfaz gráfica para administrar la base local (y también Atlas)
Datos	MongoDB Atlas	Clúster en la nube	Base de datos en la nube
Exposición	ngrok	Cuenta gratuita	Túnel HTTPS del puerto 3000 hacia Internet

3.5 Lenguajes utilizados

- **TypeScript:** aplicación móvil y backend.
- **Python:** motor de inferencia (`app/main.py`, `app/logic/evaluator.py`).
- **JavaScript (mongosh):** script de datos de prueba `database/seed.js`.
- **GraphQL SDL:** esquema generado `src/schema.gql`.

3.6 Puertos y protocolos

Tabla 18. *Puertos de comunicación*

Puerto	Servicio	Protocolo	Exposición
3000	API NestJS (`/graphql`)	HTTP, escucha en 0.0.0.0	Pública solo a través de ngrok o Nginx
443	Túnel ngrok / proxy Nginx	HTTPS (TLS)	Pública
8000	Motor IA FastAPI (`/evaluar`)	HTTP JSON	Interna (localhost)
11434	Ollama (`/api/generate`)	HTTP JSON	Interna (127.0.0.1)
27017	MongoDB local (Compass) / Atlas	Protocolo MongoDB	Local: localhost. Atlas: conexión `mongodb+srv` con lista de IP permitidas
8081	Metro bundler de Expo	HTTP	Solo red local de desarrollo

3.7 Patrones y estándares de codificación

- **Inyección de dependencias** de NestJS: los servicios reciben los modelos con `@InjectModel` y el `HttpService`.
- **DTO / InputType** para validar la entrada de cada mutación (por ejemplo, `@IsEmail` y `@MinLength(6)` en `CreateUsuarioInput`).

- **Manejo de errores uniforme:** los servicios capturan excepciones y lanzan `GraphQLError` con un código en `extensions.code` (`BAD_USER_INPUT`, `UNAUTHENTICATED`, `FORBIDDEN`, `CONFLICT`, `INTERNAL_SERVER_ERROR`).
- **Estilo:** Prettier con comillas simples y coma final (`.prettierrc`); ESLint con `typescript-eslint` en modo *recommendedTypeChecked*.
- **Nombres:** campos de base de datos y GraphQL en `snake_case` (`nombre_farmaco`); clases en `PascalCase`; funciones y variables en `camelCase`. Cada archivo inicia con un comentario `// Ruta:` que indica su ubicación.

3.8 Identidad visual y accesibilidad

Los colores y tamaños se centralizan en el hook `useAppTheme()` de `src/store/useThemeStore.ts`. Ningún componente debe usar colores fijos para texto o fondos principales; deben tomarse del tema para que el modo de alto contraste funcione.

Tabla 19. *Paleta de colores del tema*

Token	Modo normal	Alto contraste	Uso
background	#F8FAFC	#FFFFFF	Fondo de pantallas
textPrimary	#0F172A	#000000	Texto principal
textSecondary	#475569	#333333	Texto de apoyo
primary	#2563EB	#0000EE	Botones y elementos activos
success	#059669	#006600	Tomas confirmadas
danger	#E11D48	#CC0000	Botón SOS y errores
warning	#4F46E5	#660066	Citas médicas

La escala tipográfica parte de un tamaño base de 16 y se multiplica por 1, 1.2 o 1.4 según la preferencia `normal`, `large` o `xlarge` (tamaños xs 12, sm 14, base 16, lg 18, xl 20, h2 24, h1 32).

3.9 Técnicas de búsqueda, distancia y optimización de IA

Esta sección reúne siete técnicas de inteligencia artificial vistas en la asignatura y analiza cómo se aplicarían en LumaIA. Ninguna propuesta modifica dosis ni horarios prescritos: todas trabajan sobre la organización, la interacción y el monitoreo, para respetar la regla RD-01.

Definiciones

Tabla 20. *Definición de las técnicas*

Técnica	Tipo	Definición	Propiedades
BFS – Búsqueda en anchura	Búsqueda no informada	Explora el grafo por niveles: primero todos los nodos a un paso, luego los de dos pasos, y así sucesivamente, usando una cola FIFO.	Completa. Óptima en número de pasos cuando todas las acciones cuestan igual. Memoria O(b^d).
DFS – Búsqueda en profundidad	Búsqueda no informada	Avanza por una rama hasta el fondo antes de retroceder (*backtracking*), usando una pila LIFO o recursión.	Poca memoria, O(b·m). No garantiza la solución óptima; requiere control de nodos visitados en grafos con ciclos.
UCS – Búsqueda de costo uniforme	Búsqueda no informada	Expande siempre el nodo con menor costo acumulado g(n), usando una cola de prioridad. Equivale al algoritmo de Dijkstra.	Completa y óptima cuando los costos son positivos.
Hill climbing – Escalada de colinas	Búsqueda local	Parte de una solución y se mueve al vecino que mejora la función objetivo; se detiene cuando ningún vecino mejora.	Rápida y con poca memoria. Puede quedar atrapada en óptimos locales o mesetas; se mitiga con reinicios aleatorios.
Algoritmo genético	Metaheurística evolutiva	Mantiene una población de soluciones (cromosomas) que evoluciona por selección según su aptitud, cruce y mutación a lo largo de varias generaciones.	Explora espacios de búsqueda grandes. No garantiza el óptimo global y tiene alto costo de cómputo.
Distancia Manhattan	Métrica de distancia (L1)	Suma de las diferencias absolutas entre coordenadas: $d(x, y) = \Sigma \lvert x_i - y_i \rvert$.	Fácil de interpretar. Se usa como heurística en cuadrículas y para medir desviaciones acumuladas.
Distancia Euclidiana	Métrica de distancia (L2)	Distancia en línea recta entre dos puntos: $d(x, y) = \sqrt{\Sigma} (x_i - y_i)^2$.	Base de la clasificación por vecino más cercano (k-NN) y del agrupamiento (k-means).

Aplicación propuesta y viabilidad

Tabla 21. *Aplicabilidad de cada técnica en LumaIA*

Técnica	Aplicación en LumaIA	Dónde se implementa	Viabilidad
Distancia Euclidiana	Clasificar el texto libre del usuario por **vecino más cercano**. La pregunta se convierte en un vector (embeddings de Ollama o TF-IDF) y se compara con frases de ejemplo de cada intención (`PROXIMA_TOMA`, `CITAS_HOY`, `DIAGNOSTICO`, etc.). Gana la intención más cercana; si supera un umbral, se pide repetir. Corrige el caso CP-IA-10 y refuerza RD-01.	Motor IA (Python)	**Sí, implementable ahora**
Distancia Manhattan	Medir la **adherencia diaria**: suma de minutos de diferencia entre las horas programadas y las horas confirmadas. Luma o el cuidador pueden consultar el indicador ("hoy hubo 45 min de retraso acumulado").	Backend (servicio de registros de toma) o motor IA	**Sí, implementable ahora** (requiere guardar en `fecha_programada` la hora prescrita y no la de confirmación)
BFS	**Guía de navegación accesible**: las pantallas forman un grafo y cada toque es una arista. Si el usuario pregunta "¿cómo agrego una medicina?", BFS encuentra la ruta con menos toques y Luma la dicta paso a paso. También sirve para escalar una emergencia por niveles (cuidador → contactos → 911).	App móvil (TypeScript) o motor IA	**Sí, implementable ahora**
DFS	**Verificación exhaustiva del árbol de reglas** del motor: recorre en profundidad todas las ramas y comprueba que cualquier hoja alcanzable con una consulta diagnóstica (D = V) termine en bloqueo (B = V). Es la versión automatizada de las tablas de verdad de la sección 2.5.	Motor IA (`evaluator.py`) y pruebas	**Sí, implementable ahora**
UCS	**Escalamiento SOS ponderado**: la red de apoyo se modela como grafo con costo = tiempo estimado de respuesta o prioridad. UCS ordena a quién contactar primero por menor costo acumulado si alguien no responde.	Backend	**Viable con cambios**: agregar `prioridad` o `tiempo_respuesta` a los contactos y un servicio de notificaciones
Hill climbing	**Ajuste del aviso de recordatorio**: cambia en ±5 min la anticipación del aviso (nunca la hora de la toma) y conserva el cambio si reduce el retraso medido con Manhattan durante la semana.	Backend (tarea programada)	**Viable con cambios**: requiere recordatorios y notificaciones push (RF-20, pendiente) e historial suficiente
Algoritmo genético	Optimizar fuera de línea los parámetros del clasificador: qué palabras clave usa el filtro RD-01 y el umbral de distancia. Cada cromosoma es un vector binario de palabras más el umbral; la aptitud es la exactitud sobre `logs_interaccion` etiquetados, penalizando fuerte los diagnósticos no bloqueados.	Experimento externo (Python)	**No recomendable en producción por ahora**: pocos datos, alto costo de cómputo y resultados difíciles de auditar. Sí como experimento académico

Tabla 22. *Resumen de viabilidad*

Clasificación	Técnicas
Implementables ahora con la arquitectura actual	Distancia Euclidiana, Distancia Manhattan, BFS, DFS
Implementables tras agregar datos o funciones	UCS (prioridades de contacto), Hill climbing (notificaciones push)
No recomendables en producción (solo experimento)	Algoritmo genético

Bosquejos de implementación

Distancia Euclidiana – clasificación de intención (motor IA):

```
import math

def euclidiana(a, b):
    return math.sqrt(sum((x - y) ** 2 for x, y in zip(a, b)))

def clasificar(vector_pregunta, centroides, umbral=0.8):
    # centroides: {"PROXIMA_TOMA": [...], "DIAGNOSTICO": [...], ...}
    intencion, dist = min(((k, euclidiana(vector_pregunta, c)) for k, c in centroides.items()),
                          key=lambda t: t[1])
    return intencion if dist <= umbral else "NO_RECONOCIDA"
```

Distancia Manhattan – retraso acumulado de las tomas:

```
def minutos(hhmm):
    h, m = map(int, hhmm.split(":"))
    return h * 60 + m

def retraso_manhattan(programadas, confirmadas):
    return sum(abs(minutos(p) - minutos(c)) for p, c in zip(programadas, confirmadas))

retraso_manhattan(["08:00", "14:00", "20:00"], ["08:10", "14:05", "20:30"])  # 45 minutos
```

BFS – ruta con menos toques entre pantallas (app móvil):

```
const pantallas: Record<string, string[]> = {
  Inicio: ['Luma', 'Agenda', 'Apoyo'], Agenda: ['MenuAgregar'],
  MenuAgregar: ['FormMedicamento', 'FormCita', 'FormNota'], Apoyo: ['NuevoContacto'],
};

function rutaMasCorta(origen: string, destino: string): string[] | null {
  const cola: string[][] = [[origen]];
  const visitados = new Set([origen]);
  while (cola.length) {
    const ruta = cola.shift()!;
    const ultima = ruta[ruta.length - 1];
    if (ultima === destino) return ruta;
    for (const sig of pantallas[ultima] ?? []) {
      if (!visitados.has(sig)) { visitados.add(sig); cola.push([...ruta, sig]); }
    }
  }
  return null;
}
// rutaMasCorta('Inicio', 'FormMedicamento') → Inicio → Agenda → MenuAgregar → FormMedicamento
```

DFS – verificación de que toda rama diagnóstica termina en bloqueo:

```
ARBOL = {"cond": "es_diagnostico",
         "si": {"hoja": "BLOQUEO_RD01"},
         "no": {"cond": "pide_proxima_toma",
                "si": {"hoja": "PROXIMA_TOMA"}, "no": {"hoja": "NO_RECONOCIDA"}}}

def verificar_rd01(nodo, hechos=None):
    hechos = hechos or {}
    if "hoja" in nodo:   # caso base: se llegó al fondo de la rama
        return not hechos.get("es_diagnostico") or nodo["hoja"] == "BLOQUEO_RD01"
    return (verificar_rd01(nodo["si"], {**hechos, nodo["cond"]: True}) and
            verificar_rd01(nodo["no"], {**hechos, nodo["cond"]: False}))

assert verificar_rd01(ARBOL)  # D → B se cumple en todas las ramas
```

Recomendación de orden: implementar primero la Distancia Euclidiana (mejora la comprensión de Luma y la seguridad RD-01) y la Distancia Manhattan (indicador de adherencia), porque usan datos que el sistema ya almacena.

Visualización de resultados en gráficas (solo cuidador)

Los resultados de las técnicas de búsqueda y distancia se presentan 100 % en gráficas dentro de una nueva pestaña Gráficas (pantalla `PanelCuidadorScreen`). Esta pestaña y sus datos son exclusivos del rol Cuidador: el adulto mayor nunca ve gráficas; solo recibe de Luma la respuesta en texto y voz. El cuidador ve las gráficas del adulto mayor con el que está vinculado (`cuidador_vinculado_id`).

Tabla 23. *Gráfica asociada a cada técnica*

Técnica	Gráfica en el panel del cuidador	Datos que la alimentan
Distancia Euclidiana	**Barras horizontales** con la distancia de la última pregunta a cada intención y una línea de umbral; **dona** con la distribución semanal de intenciones, destacando en rojo las consultas bloqueadas por RD-01.	`logs_interaccion` (nuevos campos `intencion` y `distancia`)
Distancia Manhattan	**Línea** con los minutos de retraso acumulado por día (7 o 30 días) y **barras** del retraso promedio por medicamento.	`registros_toma` (`fecha_programada` vs `fecha_confirmacion`) y `medicamentos`
BFS	**Grafo de pantallas** (nodos y aristas) con la ruta encontrada resaltada y el número de toques; **barras** con las funciones para las que más se pidió ayuda.	Grafo de navegación de la app y `logs_interaccion` con intención de ayuda
DFS	**Barras apiladas** de ramas del árbol de reglas verificadas: ramas que terminan en bloqueo vs. ramas permitidas, y **pastel** de consultas bloqueadas vs. respondidas en la semana.	Resultado de `verificar_rd01` y `logs_interaccion.bloqueo_medico`
UCS (al implementarse)	**Barras** con el costo acumulado de cada contacto en el orden de escalamiento.	Contactos con `prioridad` / `tiempo_respuesta`
Hill climbing (al implementarse)	**Línea doble**: minutos de anticipación del aviso y retraso semanal, para ver cómo converge el ajuste.	Historial de recordatorios y `registros_toma`

Tabla 24. *Control de acceso a las gráficas*

Usuario	Pestaña Gráficas	Respuesta del API de analítica

Cuidador vinculado	Visible	Devuelve las series del adulto mayor vinculado
Cuidador sin vinculación	Visible con aviso "Vincula una cuenta para ver gráficas"	Lista vacía
Adulto mayor	No existe en su navegación	Error `FORBIDDEN`
Sin sesión	No disponible	Error `UNAUTHENTICATED`

Implementación propuesta:

1. **Backend:** crear el módulo `analitica` con queries GraphQL que devuelvan series listas para graficar, por ejemplo `adherenciaManhattan(dias: Int)`, `distribucionIntenciones(dias: Int)`, `verificacionReglas` y `rutaNavegacion(destino: String)`.
2. **Autorización por rol:** proteger el módulo con un `GqlAuthGuard` (valida el JWT) y un `RolesGuard` que solo permita `rol = CUIDADOR`. El ID del adulto mayor se obtiene del perfil del cuidador autenticado, nunca de un argumento enviado por la app. Atiende además el hallazgo S-01.
3. **Motor IA:** agregar `POST /clasificar` (devuelve la intención y la distancia a cada intención) y guardar `intencion` y `distancia` en `logs_interaccion`.
4. **App móvil:** crear `PanelCuidadorScreen` con una librería de gráficas compatible con Expo (por ejemplo `react-native-gifted-charts`, que usa `react-native-svg`, ya instalado) y registrar la pestaña **solo** cuando `rol === 'Cuidador'` en `MainNavigator`.

```
// Backend: guard de rol (src/auth/roles.guard.ts)
@Injectable()
export class RolesGuard implements CanActivate {
  canActivate(ctx: ExecutionContext): boolean {
    const req = GqlExecutionContext.create(ctx).getContext().req;
    if (req.user?.rol !== Enum_Rol.CUIDADOR) {
      throw new GraphQLError('Solo el cuidador puede ver las gráficas.', { extensions: { code:
'FORBIDDEN' } });
    }
    return true;
  }
}

// Resolver de analítica
@UseGuards(GqlAuthGuard, RolesGuard)
@Query(() => [PuntoSerie], { name: 'adherenciaManhattan' })
adherencia(@Args('dias', { type: () => Int }) dias: number, @Context() ctx) {
  return this.analiticaService.adherencia(ctx.req.user.sub, dias);
}
```

```
// App: pestaña visible solo para el cuidador (src/navigation/MainNavigator.tsx)
const { rol } = useAuthStore();
...
{rol === 'Cuidador' && (
  <Tab.Screen name="Gráficas" component={PanelCuidadorScreen}
    options={{ tabBarIcon: ({ color, size }) => <BarChart3 color={color} size={size + 4} /> }} />
)}
```

Doble protección: ocultar la pestaña en la app no basta, porque cualquiera podría llamar al API directamente. La regla se aplica también en el servidor con `RolesGuard` (regla RN-10).

Recomendación de orden: implementar primero la Distancia Euclidiana (mejora la comprensión de Luma y la seguridad RD-01) y la Distancia Manhattan (indicador de adherencia), porque usan datos que el sistema ya almacena.

4. Instalación de paqueterías requeridas

Alcance de esta sección: por indicación del docente, aquí solo se listan las paqueterías y los elementos que exigen los requisitos funcionales de la sección 2.5. No se describe el proceso completo de instalación de sistemas operativos, Node.js, Python, Git o editores, que se asumen ya disponibles.

4.1 Herramientas base que se asumen instaladas

Tabla 23. *Herramientas base*

Herramienta	Versión mínima	Motivo
Node.js y npm	20 LTS	Ejecutar NestJS 11 y Expo SDK 57
Python y pip	3.10	Ejecutar el motor IA (usa `match`/`case`)
MongoDB local + MongoDB Compass	7.x	Base de datos `lumaia` en desarrollo
Cuenta de MongoDB Atlas	Clúster gratuito o superior	Base de datos `lumaia` en la nube
Expo Go en el teléfono	Compatible con SDK 57	Probar la app en un dispositivo físico

4.2 Paquetes del backend por requisito

Tabla 24. *Paqueterías del backend y requisito que atienden*

Paquete	Requisitos que atiende
`@nestjs/graphql`, `@nestjs/apollo`, `@apollo/server`, `graphql`, `@as-integrations/express5`	Todos: exponen la API GraphQL
`@nestjs/mongoose`, `mongoose`	RF-01 a RF-11, RF-21 (persistencia)
`bcrypt`	RF-01, RF-02 (hash de contraseña)
`@nestjs/jwt`	RF-02, RF-03 (token de sesión)
`class-validator`, `class-transformer`	RF-01 (correo válido, contraseña mínima)
`@nestjs/axios`, `axios`, `rxjs`	RF-15, RF-16 (llamadas al motor IA)
`@nestjs/config`	Lectura de `MONGODB_URI` desde `.env`

Todos están declarados en `package.json`, por lo que basta con ejecutar en la raíz del repositorio del backend:

```
cd LumaIA_Backend_2.0
npm install
```

Si se parte de un proyecto vacío, la instalación equivalente es:

```
npm install @nestjs/graphql @nestjs/apollo @apollo/server graphql @as-integrations/express5 \
  @nestjs/mongoose mongoose @nestjs/jwt bcrypt class-validator class-transformer \
  @nestjs/axios axios @nestjs/config
npm install -D @types/bcrypt
```

4.3 Paquetes de la aplicación móvil por requisito

Tabla 25. *Paqueterías de la app y requisito que atienden*

Paquete	Requisitos que atiende
`@react-navigation/native`, `native-stack`, `bottom-tabs`, `react-native-screens`, `react-native-safe-area-context`	RF-12, RF-13 (navegación por pestañas)
`@tanstack/react-query`, `axios`	Todos (consumo de la API)
`zustand`, `@react-native-async-storage/async-storage`	RF-03 (sesión persistente), RF-14 (tema)
`expo-speech`	RF-15 (lectura en voz alta)
`expo-av`, `expo-file-system`	RF-17 (grabación de voz)
`@react-native-community/datetimepicker`	RF-06, RF-09 (hora de toma y fecha de cita)
`lucide-react-native`, `react-native-svg`	RF-14 (iconos grandes y legibles)

En Expo los paquetes nativos deben instalarse con `npx expo install`, que elige la versión compatible con el SDK:

```
cd LumaIA_Frontend_Movil
npm install
npx expo install expo-speech expo-av expo-file-system @react-native-community/datetimepicker \
  @react-native-async-storage/async-storage react-native-screens react-native-safe-area-context
react-native-svg
```

4.4 Paquetes del motor IA

El repositorio del motor no incluye `requirements.txt`. Se recomienda crearlo con el siguiente contenido e instalarlo en un entorno virtual:

```
# requirements.txt
fastapi
uvicorn[standard]
pydantic
```

```
cd chatbot-lumaia
python -m venv .venv
.venv\Scripts\activate        # Windows  (Linux/macOS: source .venv/bin/activate)
pip install -r requirements.txt
```

4.5 Elementos adicionales exigidos por los requisitos

Tabla 26. *Elementos adicionales*

Elemento	Requisito	Acción
Modelo Llama 3.2 en Ollama	RF-16	`ollama pull llama3.2`
Base de datos `lumaia` con datos de prueba	RF-06 a RF-10	En Compass (consola mongosh) sobre la base local, o `mongosh "<MONGODB_URI>" database/seed.js` para Atlas
Túnel HTTPS para el teléfono	Todos (app en dispositivo)	`ngrok http 3000`
Permiso de micrófono	RF-17	Se solicita en tiempo de ejecución con `Audio.requestPermissionsAsync()`

4.6 Verificación rápida

```
npm ls --depth=0          # backend y app: dependencias resueltas
npx expo-doctor           # app: compatibilidad con el SDK 57
pip show fastapi uvicorn  # motor IA
ollama list               # debe aparecer llama3.2
```

5. Estructura del proyecto

5.1 Repositorios

El código se administra con Git en tres repositorios independientes. Esta separación permite desplegar, versionar y escalar cada componente por separado.

Tabla 27. *Repositorios del sistema*

Repositorio	Contenido	Comando de arranque en desarrollo
`LumaIA_Frontend_Movil`	Aplicación Expo / React Native	`npx expo start`
`LumaIA_Backend_2.0`	API NestJS + GraphQL + Mongoose	`npm run start:dev`
`chatbot-lumaia`	Motor de inferencia FastAPI	`uvicorn app.main:app --reload --port 8000`

5.2 Backend (lumaia-backend)

```
LumaIA_Backend_2.0/
├── database/
│   └── seed.js                  Datos de prueba (mongosh)
├── src/
│   ├── asistente/               Orquestación del asistente Luma (IA)
│   ├── auth/                    Inicio de sesión y emisión de JWT
│   │   └── dto/login-response.dto.ts
│   ├── citas/                   Citas médicas (entity, dto, resolver, service)
│   ├── logs-interaccion/        Bitácora de conversaciones
│   ├── medicamentos/            Medicamentos y cálculo de horarios
│   ├── notas-personales/        Notas del usuario
│   ├── registros-toma/          Confirmación de tomas
│   ├── usuarios/                Registro, perfil, contactos y vinculación
│   ├── app.module.ts            Módulo raíz: Mongo, GraphQL y módulos de dominio
│   ├── app.resolver.ts          Query de salud lumaStatus
│   ├── main.ts                  Arranque: CORS, ValidationPipe, puerto 3000
│   └── schema.gql               Esquema GraphQL autogenerado (no editar)
├── test/                        Pruebas e2e (Jest + Supertest)
├── .prettierrc / eslint.config.mjs
├── nest-cli.json / tsconfig.json / tsconfig.build.json
└── package.json
```

Cada carpeta de dominio repite la misma estructura, lo que facilita ubicar cualquier cambio:

Tabla 28. *Estructura interna de un módulo de dominio*

Archivo	Responsabilidad
`entities/<nombre>.entity.ts`	Clase con `@Schema` (Mongoose) y `@ObjectType` (GraphQL); define campos, índices y enumeraciones.
`dto/create-<nombre>.input.ts`	Forma de los argumentos de entrada (`@InputType`) y sus validaciones.
`<nombre>.resolver.ts`	Declara las *queries* y *mutations* que ve el cliente.
`<nombre>.service.ts`	Lógica de negocio, acceso al modelo y manejo de errores.
`<nombre>.module.ts`	Registra el esquema con `MongooseModule.forFeature` y los *providers*; exporta el servicio.

5.3 Aplicación móvil (lumaia-mobile)

```
LumaIA_Frontend_Movil/
├── assets/logo_LumaIA.png
├── src/
│   ├── config/api.client.ts     Instancia Axios: URL base, timeout, token y errores
│   ├── navigation/
│   │   ├── RootNavigator.tsx    Decide entre pila Auth (Login/Register) y Main
│   │   └── MainNavigator.tsx    Pestañas: Inicio, Luma, Agenda, Apoyo
│   ├── screens/                 Login, Register, Dashboard, Chat, Agenda, Apoyo
│   ├── services/                Una función por operación GraphQL
│   ├── store/
│   │   ├── useAuthStore.ts      Sesión (token, usuarioId, rol) persistida
│   │   └── useThemeStore.ts     Contraste, tamaño de letra y paleta
│   └── types/index.ts           Interfaces Usuario, Medicamento, Cita, RegistroToma
├── App.tsx                      QueryClientProvider + NavigationContainer
├── index.ts                     registerRootComponent
├── app.json                     Configuración Expo (nombre, iconos, plugins)
└── package.json / tsconfig.json
```

5.4 Motor de inferencia (chatbot-lumaia)

```
chatbot-lumaia/
└── app/
    ├── main.py                  API FastAPI "LumaIA Engine API" con POST /evaluar (en uso)
    ├── logic/evaluator.py       Árbol de decisión evaluar_reglas() (no conectado a main.py)
    └── models/schemas.py        Modelos Pydantic de la primera versión del motor
```

Importante para mantenimiento: `main.py` redefine sus propios modelos Pydantic (`Medicamento` con `id`, `Cita`, `tomas_hoy_ids`) y no importa `evaluator.py` ni `schemas.py`. Esos dos archivos corresponden a una versión anterior; antes de modificarlos verifique si se desea reintegrarlos o eliminarlos.

6. Configuración

6.1 Variables de entorno del backend

`AppModule` carga el archivo `.env` de la raíz con `ConfigModule.forRoot({ isGlobal: true })`. El archivo `.env` está excluido de Git en `.gitignore`, por lo que cada entorno debe crear el suyo.

Tabla 29. *Variables de entorno*

Variable	Estado actual	Valor por defecto / ejemplo	Descripción
`MONGODB_URI`	Leída por `app.module.ts`	`mongodb://localhost:27017/lumaia`	Cadena de conexión: `mongodb://localhost:27017/lumaia` (local con Compass) o `mongodb+srv://…` (Atlas).
`JWT_SECRET`	Recomendada (hoy fija en `auth.module.ts`)	Cadena aleatoria de 32+ caracteres	Clave para firmar tokens. Debe moverse al `.env`.
`PORT`	Recomendada (hoy fija en `main.ts`)	`3000`	Puerto de escucha del API.
`IA_ENGINE_URL`	Recomendada (hoy fija en `asistente.service.ts`)	`http://localhost:8000/evaluar`	Endpoint del motor Luma Cloud.
`OLLAMA_URL`	Recomendada (hoy fija)	`http://127.0.0.1:11434/api/generate`	Endpoint de Ollama.
`TZ`	Recomendada	`America/Mexico_City`	Zona horaria con la que se calcula "hoy" en tomas y citas.

```
# .env (ejemplo para desarrollo)
MONGODB_URI=mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/lumaia
JWT_SECRET=cambie-esta-clave-por-una-larga-y-aleatoria
PORT=3000
IA_ENGINE_URL=http://localhost:8000/evaluar
OLLAMA_URL=http://127.0.0.1:11434/api/generate
TZ=America/Mexico_City
```

6.2 Configuración del servidor GraphQL

Tabla 30. *Parámetros de GraphQLModule y main.ts*

Parámetro	Valor	Efecto
`driver`	`ApolloDriver`	Usa Apollo Server 5 como motor GraphQL.
`autoSchemaFile`	`src/schema.gql`	Regenera el esquema en cada arranque (code-first).
`sortSchema`	`true`	Ordena alfabéticamente tipos y campos.
`playground` / `plugins`	`false` / Landing Page local	Habilita Apollo Sandbox en `http://localhost:3000/graphql`.
`app.enableCors()`	Sin restricciones	Permite peticiones desde la app web y Expo.
`ValidationPipe` global	Activado	Aplica las reglas de `class-validator` de los DTOs.
`app.listen(3000, '0.0.0.0')`	Todas las interfaces	Permite acceso desde la red local y el túnel.

6.3 Configuración de la aplicación móvil

La URL del backend se define en `src/config/api.client.ts`. Cada vez que cambie el túnel ngrok o el dominio de producción, este es el único archivo que debe editarse:

```
export const apiClient = axios.create({
  baseURL: 'https://<su-subdominio>.ngrok-free.dev/graphql', // sin espacios al inicio
  timeout: 45000,          // 45 s para permitir respuestas de Ollama
  headers: { 'Content-Type': 'application/json' },
});
```

Atención: en la versión actual el valor de `baseURL` inicia con dos espacios (`'  https://...'`). Se recomienda eliminarlos para evitar errores de URL inválida en algunas plataformas.

Tabla 31. *Otros parámetros de la app*

Archivo	Parámetro	Valor
`api.client.ts`	Interceptor de petición	Agrega `Authorization: Bearer <token>` si hay sesión
`api.client.ts`	Interceptor de respuesta	Traduce *timeout* y falta de red a mensajes en español
`App.tsx`	QueryClient	`retry: 1`, `refetchOnWindowFocus: false`
`useAuthStore.ts`	Clave de almacenamiento	`luma-auth-storage` (AsyncStorage)
`app.json`	Nombre / slug / versión	`lumaia-mobile` / `lumaia-mobile` / `1.0.0`
`app.json`	Orientación y plugins	`portrait`; plugin `@react-native-community/datetimepicker`
`tsconfig.json`	Compilador	Extiende `expo/tsconfig.base` con `strict: true`

6.4 Configuración del motor IA y de Ollama

```
# Motor Luma Cloud (desde la carpeta chatbot-lumaia)
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# Ollama (servicio local)
ollama serve
ollama pull llama3.2
```

Para el modo local, el backend envía a Ollama `{ model: "llama3.2", prompt, format: "json", stream: false }` y espera un objeto JSON con las claves `respuesta_texto` e `intencion`. El *system prompt* se construye en `AsistenteService.procesarPregunta` y es el lugar donde deben ajustarse las reglas de comportamiento del modelo.

6.5 Base de datos

El equipo trabaja con dos entornos de MongoDB que comparten el mismo esquema:

Tabla 32. *Entornos de base de datos*

Entorno	Herramienta	Cadena de conexión (MONGODB_URI)
Local (desarrollo)	MongoDB Community Server administrado con MongoDB Compass	`mongodb://localhost:27017/lumaia`
Nube (compartido / producción)	MongoDB Atlas	`mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/lumaia`

- En Compass, cree una conexión a `mongodb://localhost:27017` y la base `lumaia`. Las colecciones se crean automáticamente al insertar el primer documento.
- Compass también puede conectarse al clúster de Atlas pegando la cadena `mongodb+srv` para revisar los datos de la nube.
- Mongoose crea los índices (correo único y TTL) al arrancar el backend (`autoIndex` activo por defecto).
- Para cargar datos de prueba, abra la consola **mongosh** integrada en Compass sobre la base `lumaia` y pegue el contenido de `database/seed.js`, o ejecute `mongosh "<MONGODB_URI>" database/seed.js`. El script **borra** primero el contenido de las seis colecciones.
- En MongoDB Atlas, agregue la IP del servidor (o 0.0.0.0/0 solo en desarrollo) en *Network Access*.

6.6 Herramientas de calidad de código

Tabla 33. *Archivos de configuración de calidad*

Archivo	Configuración
`.prettierrc`	`singleQuote: true`, `trailingComma: "all"`
`eslint.config.mjs`	ESLint 9 + `typescript-eslint` (recommendedTypeChecked) + Prettier; `no-explicit-any` desactivada; `no-floating-promises` como advertencia
`nest-cli.json`	`sourceRoot: "src"`, `deleteOutDir: true` (limpia `dist` al compilar)
`tsconfig.json` (backend)	Destino ES2021, CommonJS, decoradores habilitados, `strictNullChecks: false`

7. Modelo de datos

7.1 Enfoque

LumaIA usa **MongoDB**, una base de datos orientada a documentos, en local (administrada con MongoDB Compass) y en la nube (MongoDB Atlas). Las relaciones entre colecciones se resuelven con **referencias por ObjectId** (por ejemplo, `medicamentos.usuario_id → usuarios._id`) y los contactos de emergencia se guardan **embebidos** dentro del documento del usuario, porque siempre se consultan junto con él. Las colecciones con `timestamps: true` agregan automáticamente `createdAt` y `updatedAt`.

7.2 Relaciones entre colecciones

Tabla 34. *Relaciones entre colecciones*

Colección origen	Campo	Colección destino	Cardinalidad	Tipo de relación
medicamentos	`usuario_id`	usuarios	Muchos a uno (N:1)	Referencia por ObjectId
citas	`usuario_id`	usuarios	Muchos a uno (N:1)	Referencia por ObjectId
notas_personales	`usuario_id`	usuarios	Muchos a uno (N:1)	Referencia por ObjectId
logs_interaccion	`usuario_id`	usuarios	Muchos a uno (N:1)	Referencia por ObjectId
registros_toma	`medicamento_id`	medicamentos	Muchos a uno (N:1)	Referencia por ObjectId
usuarios	`cuidador_vinculado_id`	usuarios	Uno a uno (1:1)	Autorreferencia (cuidador ↔ adulto mayor)
usuarios	`contactos_emergencia`	—	Uno a muchos (1:N)	Subdocumentos embebidos

7.3 Diccionario de datos

Las tablas siguientes describen cada colección con las restricciones exactas definidas en las entidades del código.

Tabla 35. *Colección usuarios*

Campo	Tipo	Restricciones	Descripción
`_id`	ObjectId	PK	Identificador del usuario
`nombre`	String	Requerido	Nombre completo
`correo`	String	Requerido, único	Credencial de acceso
`password_hash`	String	Requerido	Hash bcrypt de la contraseña
`rol`	Enum_Rol	Requerido	`Adulto Mayor` o `Cuidador`
`cuidador_vinculado_id`	ObjectId	FK → usuarios, opcional	Usuario vinculado (en ambos sentidos)

Campo	Tipo	Restricciones	Descripción
pin_vinculacion	String	Opcional, 6 dígitos	PIN temporal; se elimina al usarse
estado_activo	Boolean	Default true	Borrado lógico de la cuenta
tokens_dispositivo	String[]	Default []	Tokens push (reservado)
contactos_emergencia	Array embebido	Default []	{ nombre_contacto, telefono, parentesco }

Tabla 36. *Colección medicamentos*

Campo	Tipo	Restricciones	Descripción
_id	ObjectId	PK	Identificador
usuario_id	ObjectId	FK → usuarios, requerido	Dueño del tratamiento
nombre_farmaco	String	Requerido	Nombre comercial
dosis	String	Requerido	Texto libre, p. ej. "850mg"
frecuencia_horas	Int	Opcional	Intervalo entre tomas
horarios_especificos	String[]	Default [], formato HH:mm	Horarios calculados (RN-03)
fecha_eliminacion	Date	Opcional, índice TTL 30 días	Marca de borrado lógico

Tabla 37. *Colección citas*

Campo	Tipo	Restricciones	Descripción
_id	ObjectId	PK	Identificador
usuario_id	ObjectId	FK → usuarios, requerido	Dueño de la cita
titulo_evento	String	Requerido	Motivo de la cita
fecha_hora	Date	Requerido (UTC)	Fecha y hora programada
ubicacion	String	Opcional	Clínica o consultorio
estado	EnumEstadoCita	Default Programada	Programada, Completada, Cancelada
fecha_eliminacion	Date	Opcional, índice TTL 30 días	Marca de borrado lógico

Tabla 38. *Colección registros_toma*

Campo	Tipo	Restricciones	Descripción
_id	ObjectId	PK	Identificador
medicamento_id	ObjectId	FK → medicamentos, requerido	Medicamento tomado
fecha_programada	Date	Requerido	Hoy se guarda la fecha de confirmación
estado_toma	Enum_EstadoToma	Default Pendiente	Pendiente, Confirmada, Omitida
fecha_confirmacion	Date	Opcional	Momento en que se pulsó "Ya lo tomé"

Tabla 39. *Colección notas_personales*

Campo	Tipo	Restricciones	Descripción
`_id`	ObjectId	PK	Identificador
`usuario_id`	ObjectId	FK → usuarios, requerido	Autor de la nota
`contenido`	String	Requerido	Texto de la nota
`fecha_creacion`	Date	Timestamp automático	Sustituye a `createdAt`
`fecha_eliminacion`	Date	Opcional, índice TTL 30 días	Marca de borrado lógico

Tabla 40. *Colección logs_interaccion*

Campo	Tipo	Restricciones	Descripción
`_id`	ObjectId	PK	Identificador
`usuario_id`	ObjectId	FK → usuarios, requerido	Usuario que preguntó
`premisa_entrada`	String	Requerido	Pregunta o código enviado
`respuesta_generada`	String	Requerido	Texto devuelto por el motor
`bloqueo_medico`	Boolean	Default `false`	Indica si se aplicó RD-01
`fecha_interaccion`	Date	Default `Date.now`	Momento de la consulta

7.4 Enumeraciones

Las enumeraciones se guardan en MongoDB con su **valor** (por ejemplo `"Adulto Mayor"`), pero en GraphQL viajan con su **nombre** (`ADULTO_MAYOR`). La app traduce entre ambos con `mapEnumToGraphQLRole` y `mapGraphQLRoleToEnum` en `auth.service.ts`.

Tabla 41. *Enumeraciones del sistema*

Enumeración	Nombre en GraphQL	Valor en MongoDB
`Enum_Rol`	ADULTO_MAYOR · CUIDADOR	Adulto Mayor · Cuidador
`EnumEstadoCita`	PROGRAMADA · COMPLETADA · CANCELADA	Programada · Completada · Cancelada
`Enum_EstadoToma`	PENDIENTE · CONFIRMADA · OMITIDA	Pendiente · Confirmada · Omitida

7.5 Índices

Tabla 42. *Índices definidos*

Colección	Campo	Tipo	Propósito
usuarios	`correo`	Único	Impide cuentas duplicadas (error 11000 → `CONFLICT`)
medicamentos, citas, notas_personales	`fecha_eliminacion`	TTL, `expireAfterSeconds: 2592000`	Eliminación definitiva 30 días después del borrado lógico (RD-02)

Recomendación: crear índices sobre `usuario_id` en `medicamentos`, `citas` y `notas_personales`, y sobre `{ medicamento_id, fecha_confirmacion }` en `registros_toma`, ya que todas las consultas filtran por esos campos.

7.6 Datos de prueba

El script `database/seed.js` genera un escenario coherente para pruebas: un adulto mayor (Pedro) con dos contactos de emergencia, dos cuidadores, dos medicamentos (Losartán cada 12 h a las 08:00 y 20:00; Metformina 850 mg a las 14:00), una cita, una nota, dos registros de toma y dos interacciones con el asistente, una de ellas con `bloqueo_medico: true`. El detalle está en el Anexo B.

8. Módulos y API

8.1 Módulos del backend

Tabla 43. *Módulos NestJS registrados en AppModule*

Módulo	Responsabilidad	Colecciones que usa
`UsuariosModule`	Registro, perfil, contactos de emergencia, PIN y vinculación	usuarios
`AuthModule`	Validación de credenciales y firma de JWT (30 días)	usuarios
`MedicamentosModule`	Alta y consulta de medicamentos; cálculo de horarios	medicamentos
`CitasModule`	Alta y consulta de citas	citas
`NotasPersonalesModule`	Alta y consulta de notas	notas_personales
`RegistrosTomaModule`	Confirmación de tomas y tomas del día	registros_toma, medicamentos
`LogsInteraccionModule`	Esquema de bitácora (query base vacía)	logs_interaccion
`AsistenteModule`	Contexto del día, enrutamiento al motor IA y auditoría	medicamentos, citas, registros_toma, logs_interaccion

8.2 Endpoint y forma de consumo

Toda la comunicación de la app con el servidor se hace mediante `POST /graphql` con un cuerpo JSON `{ "query": "...", "variables": { ... } }`. En desarrollo, al abrir `http://localhost:3000/graphql` en el navegador aparece **Apollo Sandbox**, desde donde se pueden explorar el esquema y ejecutar operaciones.

8.3 Queries (lectura)

Tabla 44. *Queries disponibles*

Query	Argumentos	Retorna	Descripción
`lumaStatus`	—	String	Verificación de salud del API
`obtenerUsuario`	`usuario_id: ID!`	Usuario	Perfil con contactos de emergencia
`obtenerMedicamentosPorUsuario`	`usuario_id: ID!`	[Medicamento]	Medicamentos sin borrado lógico
`obtenerCitasPorUsuario`	`usuario_id: ID!`	[Cita]	Citas sin borrado lógico
`obtenerNotasPersonalesPorUsuario`	`usuario_id: ID!`	[NotaPersonal]	Notas sin borrado lógico
`obtenerTomasHoy`	`usuario_id: ID!`	[RegistroToma]	Tomas confirmadas entre 00:00 y 23:59 de hoy
`registrosTomaBase`, `logsInteraccionBase`	—	Lista vacía	Marcadores de posición (sin lógica)

8.4 Mutations (escritura)

Tabla 45. *Mutations disponibles*

Mutation	Argumentos	Retorna	Descripción / errores
`registrarUsuario`	`input: CreateUsuarioInput!`	Usuario	Hash bcrypt. Correo duplicado → `CONFLICT`
`login`	`correo`, `password`	LoginResponse	`access_token`, `usuario_id`, `rol`. Error → `UNAUTHENTICATED` / `FORBIDDEN`
`agregarContactoEmergencia`	`input: AgregarContactoInput!`	Usuario	Agrega con `$push` al arreglo embebido
`generarPinVinculacion`	`usuario_id: ID!`	String	PIN de 6 dígitos guardado en el usuario
`vincularCuidador`	`input: VincularCuidadorInput!`	Usuario	Vincula ambos usuarios y borra el PIN. PIN inválido → `BAD_USER_INPUT`
`crearMedicamento`	`createMedicamentoInput`	Medicamento	Calcula `horarios_especificos` (RN-03)
`confirmarTomaMedicamento`	`medicamento_id: ID!`	RegistroToma	Crea registro con estado `CONFIRMADA`
`crearCita`	`createCitaInput`	Cita	Estado inicial `PROGRAMADA`
`crearNotaPersonal`	`createNotaPersonalInput`	NotaPersonal	Guarda la nota del usuario
`consultarAsistente`	`usuario_id`, `pregunta_id`, `modelo`	AsistenteResponse	`respuesta_texto` e `intencion`; `modelo` por defecto `"Luma Cloud"`

8.5 Ejemplos de operaciones

Inicio de sesión

```
mutation Login($correo: String!, $password: String!) {
  login(correo: $correo, password: $password) { access_token usuario_id rol }
}
# Respuesta
{ "data": { "login": { "access_token": "eyJhbGciOi...", "usuario_id": "6721f0...", "rol":
"ADULTO_MAYOR" } } }
```

Alta de medicamento con cálculo de horarios

Si se registra un medicamento cada 8 horas con primera toma a las 08:30, el servicio genera [24/8] = 3 horarios: 08:30, 16:30 y 00:30.

```
mutation {
  crearMedicamento(createMedicamentoInput: {
    usuario_id: "6721f0...", nombre_farmaco: "Paracetamol", dosis: "500mg",
    frecuencia_horas: 8, horarios_especificos: ["08:30"]
  }) { _id horarios_especificos }
}
# horarios_especificos: ["08:30", "16:30", "00:30"]
```

Consulta al asistente

```
mutation ConsultarAsistente($usuario_id: ID!, $pregunta_id: String!, $modelo: String) {
  consultarAsistente(usuario_id: $usuario_id, pregunta_id: $pregunta_id, modelo: $modelo) {
    respuesta_texto
    intencion
  }
}
```

Compatibilidad: el resolver devuelve el objeto `AsistenteResponse`, por lo que la consulta debe pedir los subcampos `respuesta_texto` e `intencion`. El archivo `ia.service.ts` de la app aún pide `consultarAsistente(...)` sin subcampos (como cuando devolvía un `String`); debe actualizarse como se muestra arriba (ver ERR-05).

8.6 API del motor de inferencia (FastAPI)

Tabla 46. *Endpoint del motor IA*

Elemento	Detalle
Método y ruta	`POST http://localhost:8000/evaluar`
Documentación interactiva	`http://localhost:8000/docs` (Swagger generado por FastAPI)
Cuerpo	`pregunta_id: str`, `medicamentos_activos: [{ id, nombre_farmaco, horarios_especificos[] }]`, `citas: [{ titulo_evento, fecha_hora, ubicacion }]`, `tomas_hoy_ids: [str]`. El backend envía además `prompt_forzado`, que el motor ignora.
Respuesta	`{ "respuesta": "<texto>" }` con código 200; 422 si falta `pregunta_id`; 500 ante excepción

Tabla 47. *Códigos de pregunta que reconoce el motor (modo Luma Cloud)*

pregunta_id	Origen en la app	Comportamiento
`PROXIMA_TOMA`	Botón "Próxima Medicina"	Lista los medicamentos aún no tomados hoy con sus horarios; felicita si no queda ninguno.
`TOMAS_REALIZADAS`	Botón "Ya Tomadas"	Enumera los medicamentos confirmados hoy.
`CITAS_HOY`	Botón "Citas Hoy"	Lista título, hora y ubicación de las citas del día.
`RESUMEN_DIA`	Botón "Resumen del Día"	Reporte en tres partes: citas, ya tomados y pendientes.
Contiene `SINTOMA` o `DOLOR`	—	Respuesta de bloqueo RD-01 que remite al botón SOS.
Cualquier otro texto	Texto libre del usuario	"¿Puedes repetirlo? Estoy aquí para ayudarte."

8.7 Lógica del AsistenteService

8. **Extracción de contexto:** consulta medicamentos activos, citas de hoy y tomas confirmadas de hoy del usuario.
9. **Cruce de datos:** separa `medsPendientes` y `medsTomados` comparando los `_id` como texto.

10. **System prompt:** construye las instrucciones para el modelo (tono cálido, respuestas breves, salida JSON con `respuesta_texto` e `intencion`).
11. **Clasificación rápida de intención:** si el texto contiene "agregar", "nuevo", "crear" o "registrar" junto con palabras como "pastilla", "cita" o "contacto", asigna `ACCION_NUEVO_MEDICAMENTO`, `ACCION_NUEVA_CITA` o `ACCION_NUEVO_CONTACTO`; si no, `CONSULTA_GENERAL`.
12. **Enrutamiento:** si `modelo` es `"Llama 3.2 Local"` llama a Ollama; en cualquier otro caso llama al motor FastAPI.
13. **Normalización:** `parsearRespuestaJSON` limpia bloques de código y comillas; si el texto no es JSON válido lo usa tal cual con la intención de respaldo.
14. **Auditoría:** guarda la interacción en `logs_interaccion` y devuelve la respuesta.

8.8 Capa de servicios de la aplicación

Tabla 48. *Servicios del frontend*

Archivo	Operaciones GraphQL	Pantallas que lo usan
`auth.service.ts`	`login`, `registrarUsuario`	LoginScreen, RegisterScreen
`medicamentos.service.ts`	`obtenerMedicamentosPorUsuario`, `crearMedicamento`	DashboardScreen, AgendaScreen
`citas.service.ts`	`obtenerCitasPorUsuario`, `crearCita`	DashboardScreen, AgendaScreen
`notas.service.ts`	`obtenerNotasPersonalesPorUsuario`, `crearNotaPersonal`	AgendaScreen
`registros-toma.service.ts`	`obtenerTomasHoy`, `confirmarTomaMedicamento`	DashboardScreen
`contactos.service.ts`, `usuarios.service.ts`	`obtenerUsuario`, `agregarContactoEmergencia`	ApoyoScreen
`ia.service.ts`	`consultarAsistente`	ChatScreen

8.9 Navegación y pantallas

Tabla 49. *Pantallas de la aplicación*

Pantalla	Pestaña	Función
LoginScreen	— (pila Auth)	Inicio de sesión; guarda token, usuario y rol en `useAuthStore`.
RegisterScreen	— (pila Auth)	Alta con selección de rol y validación de contraseña.
DashboardScreen	Inicio	Medicamentos con botón "Ya lo tomé", citas de hoy, acceso al SOS y cierre de sesión.
ChatScreen	Luma	Chat con 4 botones rápidos, texto libre, micrófono, selector de motor y lectura en voz.
AgendaScreen	Agenda	Lista filtrable y botón flotante para crear medicamento, cita o nota.
ApoyoScreen	Apoyo	Botón de llamada de emergencia, cuidador vinculado y contactos adicionales.

9. Pruebas

9.1 Estrategia de pruebas

Tabla 50. *Niveles de prueba*

Nivel	Herramienta	Objetivo
Unitarias	Jest + ts-jest (backend)	Verificar clases y servicios de forma aislada.
Extremo a extremo (e2e)	Jest + Supertest	Levantar la aplicación Nest y probar peticiones HTTP reales.
API	Apollo Sandbox (`/graphql`) y Swagger (`/docs`)	Validar cada query, mutation y el endpoint `/evaluar`.
Funcionales	Expo Go en dispositivo físico	Recorrer los flujos de usuario de punta a punta.
Reglas lógicas	Tablas de verdad (sección 2.5)	Usar las tablas como oráculo de resultados esperados.
Estáticas	ESLint, `tsc --noEmit`, `expo lint`	Detectar errores de tipos y estilo antes de ejecutar.

9.2 Comandos

```
# Backend
npm run test          # unitarias
npm run test:e2e      # extremo a extremo
npm run test:cov      # cobertura (carpeta coverage/)
npm run lint          # ESLint con corrección automática

# App móvil
npx tsc --noEmit      # verificación de tipos
npx expo lint

# Motor IA (con el servidor activo)
curl -X POST http://localhost:8000/evaluar -H "Content-Type: application/json" \
     -d "{\"pregunta_id\":\"CITAS_HOY\",\"medicamentos_activos\":[],\"citas\":[]}"
```

9.3 Pruebas automatizadas existentes

Tabla 51. *Estado de las pruebas incluidas en el repositorio*

Archivo	Qué prueba	Observación
`src/app.controller.spec.ts`	Que `AppController.getHello()` devuelva "Hello World!"	Crea su propio módulo de prueba, por lo que se ejecuta de forma independiente. Es la prueba por defecto de NestJS y no cubre lógica de LumaIA.
`test/app.e2e-spec.ts`	Que `GET /` responda 200 "Hello World!"	Fallará: `AppController` no está registrado en `AppModule` (solo `AppResolver`) y la prueba requiere una conexión activa a MongoDB. Debe reescribirse para consultar `lumaStatus` por GraphQL.

Recomendación: agregar pruebas unitarias para `MedicamentosService.create` (cálculo de horarios), `AuthService.validarUsuario` (credenciales y cuenta inactiva) y `AsistenteService.parsearRespuestaJSON`, usando modelos Mongoose simulados (*mocks*).

9.4 Pruebas ejecutadas al motor IA

Los siguientes casos se ejecutaron el 27 de octubre de 2026 contra el código de `app/main.py` con el cliente de pruebas de FastAPI, usando los medicamentos del script de datos de prueba (Losartán 08:00 y 20:00 con id `m1`; Metformina 14:00 con id `m2`) y una cita "Revisión General" a las 10:00.

Tabla 52. *Casos de prueba del endpoint /evaluar*

ID	Entrada	Resultado obtenido	Veredicto
CP-IA-01	`PROXIMA_TOMA`, tomado: m1	"Aún tienes pendiente tomar: Metformina a las 14:00."	Correcto
CP-IA-02	`PROXIMA_TOMA`, sin medicamentos	"No tienes ningún medicamento registrado en tu rutina."	Correcto
CP-IA-03	`PROXIMA_TOMA`, tomados: m1 y m2	"¡Felicidades! Ya te tomaste todas tus medicinas por hoy."	Correcto
CP-IA-04	`TOMAS_REALIZADAS`, sin tomas	"Aún no te has tomado ninguna medicina el día de hoy…"	Correcto
CP-IA-05	`TOMAS_REALIZADAS`, tomado: m1	"¡Muy bien! Hoy ya te tomaste: Losartán…"	Correcto
CP-IA-06	`CITAS_HOY`, una cita	"Hoy tienes: Revisión General a las 10:00 en Clínica del Centro."	Correcto
CP-IA-07	`CITAS_HOY`, sin citas	"No tienes citas médicas en tu agenda para el día de hoy."	Correcto
CP-IA-08	`RESUMEN_DIA`, cita + m1 tomado	Resumen con citas, "Ya tomaste: Losartán" y "Te falta tomar: Metformina a las 14:00"	Correcto
CP-IA-09	`DOLOR_PECHO`	"Por tu seguridad, no puedo dar diagnósticos. Por favor, presiona el botón SOS."	Correcto (RD-01)
CP-IA-10	Texto libre "me duele el pecho"	"¿Puedes repetirlo? Estoy aquí para ayudarte."	Hallazgo: RD-01 no se aplica a texto libre en minúsculas
CP-IA-11	Cuerpo sin `pregunta_id`	HTTP 422 (validación Pydantic)	Correcto
CP-IA-12	`evaluar_reglas` con `DIAGNOSTICO`	"Por seguridad, no puedo emitir diagnósticos ni alterar tratamientos…"	Correcto (módulo no conectado)

Observación: `PROXIMA_TOMA` lista todos los medicamentos pendientes con todos sus horarios, sin filtrar por la hora actual. El algoritmo que busca la toma más próxima existe en `evaluator.py`, pero no se usa.

9.5 Casos de prueba de la API GraphQL

Tabla 53. *Casos de prueba de la API (resultado esperado según el código)*

ID	Operación y datos	Resultado esperado
CP-API-01	`registrarUsuario` con datos válidos	Usuario creado con `estado_activo: true` y `password_hash` en formato bcrypt

ID	Operación y datos	Resultado esperado
CP-API-02	`registrarUsuario` con correo existente	Error "El correo electrónico ya está registrado." (`CONFLICT`)
CP-API-03	`registrarUsuario` con contraseña de 5 caracteres	Error de validación "La contraseña debe tener al menos 6 caracteres"
CP-API-04	`login` con contraseña incorrecta	Error "Correo o contraseña incorrectos." (`UNAUTHENTICATED`)
CP-API-05	`login` de cuenta con `estado_activo: false`	Error "Esta cuenta se encuentra desactivada." (`FORBIDDEN`)
CP-API-06	`crearMedicamento` cada 8 h desde 08:30	`horarios_especificos = ["08:30","16:30","00:30"]`
CP-API-07	`confirmarTomaMedicamento` y luego `obtenerTomasHoy`	El registro aparece con `estado_toma: CONFIRMADA`
CP-API-08	`vincularCuidador` con PIN inexistente	Error "El PIN ingresado es inválido o ya expiró."
CP-API-09	`generarPinVinculacion` + `vincularCuidador` con PIN correcto	Ambos usuarios quedan vinculados y el PIN se elimina
CP-API-10	`obtenerCitasPorUsuario` con un ID mal formado	Error "Error al consultar la agenda..." (`INTERNAL_SERVER_ERROR`)
CP-API-11	`consultarAsistente` con selección `{ respuesta_texto intencion }`	Respuesta del motor y nuevo documento en `logs_interaccion`

9.6 Casos de prueba funcionales en la app

Tabla 54. *Casos de prueba funcionales*

ID	Pasos	Resultado esperado
CP-UI-01	Registrar cuenta con contraseñas distintas	Alerta "Las contraseñas no coinciden."
CP-UI-02	Registrar e iniciar sesión; cerrar y reabrir la app	La sesión se conserva (AsyncStorage)
CP-UI-03	En Inicio pulsar "Ya lo tomé"	La tarjeta cambia a verde, texto tachado y botón "Toma Confirmada"
CP-UI-04	En Agenda crear un medicamento, una cita y una nota	Mensaje de éxito y aparición inmediata en la lista
CP-UI-05	En Apoyo agregar contacto y pulsar "LLAMADA DE EMERGENCIA"	Se abre el marcador con el número del primer contacto
CP-UI-06	En Luma pulsar "Resumen del Día"	Burbuja con el resumen y lectura en voz es-MX
CP-UI-07	Cambiar el motor a "Llama 3.2 Local" y preguntar	Respuesta generada por Ollama en menos de 45 s
CP-UI-08	Apagar el backend y abrir Agenda	"No se pudo conectar con el servidor. Verifica tu red."

10. Despliegue y mantenimiento

10.1 Despliegue en entorno de desarrollo

El orden recomendado de arranque es de la capa de datos hacia la de presentación. Cada comando se ejecuta en una terminal distinta:

```
# 1. Base de datos: verificar que el servicio MongoDB local esté activo y conectarse con
#    MongoDB Compass a mongodb://localhost:27017 (o usar la cadena de Atlas en .env)

# 2. Motor IA Luma Cloud
cd chatbot-lumaia && uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 3. (Opcional) Modelo local
ollama serve

# 4. Backend
cd LumaIA_Backend_2.0 && npm run start:dev

# 5. Túnel HTTPS (copiar la URL en src/config/api.client.ts)
ngrok http 3000

# 6. App móvil (escanear el QR con Expo Go)
cd LumaIA_Frontend_Movil && npx expo start
```

10.2 Despliegue en producción

Tabla 55. *Nodos de despliegue recomendados*

Nodo	Software	Procedimiento
Servidor de aplicación (Ubuntu Server 22.04/24.04 LTS)	Node.js 20 LTS, PM2	`npm ci && npm run build` y `pm2 start dist/main.js --name lumaia-api`; `pm2 save` y `pm2 startup` para reinicio automático.
Mismo servidor o uno dedicado	Python 3.10+, Uvicorn	`pm2 start "uvicorn app.main:app --host 127.0.0.1 --port 8000 --workers 2" --name lumaia-ia` o un servicio *systemd*.
Servidor con GPU o CPU suficiente (opcional)	Ollama + `llama3.2`	Ejecutar `ollama serve` como servicio y precargar el modelo.
Proxy inverso	Nginx + certificado Let's Encrypt	Publicar 443 y redirigir `/graphql` a `http://127.0.0.1:3000`. El puerto 80 redirige a 443.
Base de datos	MongoDB Atlas (administrable también desde Compass)	Clúster con usuario de mínimos privilegios y lista de IP restringida a los servidores.
Distribución de la app	EAS Build	Actualizar `baseURL` al dominio de producción y ejecutar `npx eas-cli@latest build -p android --profile production` (.aab) o `--profile preview` (.apk).

```
# /etc/nginx/sites-available/lumaia (fragmento)
server {
    listen 443 ssl;
    server_name api.lumaia.example;
    location /graphql {
        proxy_pass http://127.0.0.1:3000/graphql;
        proxy_set_header Host $host;
        proxy_read_timeout 60s;   # respuestas de Ollama
    }
}
```

10.3 Mantenimiento preventivo

Tabla 56. *Tareas de mantenimiento*

Tarea	Frecuencia	Acción / comando
Respaldo de la base de datos	Diaria	Atlas: respaldos automáticos del clúster. Local: exportar colecciones desde Compass (*Export Collection*) o `mongodump --uri "$MONGODB_URI"`
Revisión de vulnerabilidades	Mensual	`npm audit` en backend y app; `pip list --outdated` en el motor
Actualización de Expo	Con cada SDK nuevo	`npx expo install expo@latest` y `npx expo install --fix`; `npx expo-doctor`
Revisión de la bitácora del asistente	Semanal	Buscar en `logs_interaccion` preguntas no reconocidas o con síntomas para mejorar reglas
Verificación de índices TTL	Después de cada despliegue	`db.medicamentos.getIndexes()` debe mostrar `expireAfterSeconds: 2592000`
Rotación de secretos	Semestral o ante incidente	Cambiar `JWT_SECRET` y contraseña de Atlas; reiniciar con `pm2 restart all`
Monitoreo de procesos	Continua	`pm2 status`, `pm2 logs lumaia-api`
Actualización del modelo local	Trimestral	`ollama pull llama3.2`

10.4 Procedimiento para agregar una funcionalidad

1. Generar el recurso: `npx nest g resource <nombre>` y elegir *GraphQL (code first)*.
2. Definir la entidad con `@Schema` y `@ObjectType`, el DTO con `@InputType` y sus validaciones.
3. Registrar el esquema en el módulo con `MongooseModule.forFeature` y el módulo en `AppModule`.
4. Arrancar el backend para regenerar `schema.gql` y probar la operación en Apollo Sandbox.
5. En la app, crear `src/services/<nombre>.service.ts`, agregar el tipo en `types/index.ts` y consumirlo con `useQuery`/`useMutation`, invalidando la `queryKey` correspondiente.
6. Actualizar este manual (secciones 2.5, 7 y 8) y el control de versiones.

10.5 Control de versiones del código

- Rama `main` estable; ramas `feature/<descripcion>` y `fix/<descripcion>` para cambios.
- Mensajes de commit en imperativo y en español, por ejemplo: "Agrega guard JWT a resolvers de medicamentos".

- Nunca subir `.env`, llaves `.jks`, `.p8` o `.p12` (ya excluidas en `.gitignore`).

11. Seguridad

11.1 Controles implementados

Tabla 57. *Controles de seguridad presentes en el código*

Control	Implementación
Hash de contraseñas	`bcrypt.genSalt(10)` + `bcrypt.hash`; comparación con `bcrypt.compare` en el inicio de sesión.
Mensaje de error genérico	Correo inexistente y contraseña incorrecta devuelven el mismo mensaje, lo que evita enumerar cuentas.
Token de sesión	JWT firmado con payload `{ sub, rol }` y expiración de 30 días; enviado como `Bearer` por la app.
Validación de entrada	`ValidationPipe` global con `@IsEmail` y `@MinLength(6)`; Pydantic en el motor IA (HTTP 422).
Cuentas desactivadas	Si `estado_activo` es `false` el inicio de sesión responde `FORBIDDEN`.
PIN de un solo uso	El PIN se elimina con `$unset` al completar la vinculación.
Transporte cifrado	La app se comunica por HTTPS mediante ngrok (desarrollo) o Nginx con TLS (producción).
Restricción ética RD-01	Respuesta de bloqueo para códigos de síntoma/dolor en `main.py`; la regla también está redactada en el *system prompt* preparado para un LLM en la nube.
Procesamiento privado	Opción de modelo local (Ollama) para que las consultas no salgan del servidor.
Auditoría	Toda consulta al asistente queda registrada en `logs_interaccion`.

11.2 Hallazgos de la revisión de código

Durante la elaboración de este manual se revisó el código fuente y se identificaron los siguientes riesgos. Se listan por prioridad para orientar el mantenimiento correctivo.

Tabla 58. *Hallazgos de seguridad y recomendaciones*

ID	Hallazgo	Riesgo	Recomendación	Prioridad
S-01	Los resolvers no validan el JWT; reciben `usuario_id` como argumento.	Cualquier cliente que conozca un ID puede leer o escribir datos de otro usuario.	Crear un `GqlAuthGuard` con `JwtService.verifyAsync` y obtener el usuario del token (`sub`).	Alta
S-02	`password_hash` está marcado con `@Field` en la entidad `Usuario`.	El hash puede consultarse por GraphQL.	Quitar el decorador `@Field` de ese campo.	Alta
S-03	La clave JWT está escrita en `auth.module.ts`.	Quien vea el repositorio puede falsificar tokens.	Leer `JWT_SECRET` desde `.env` con `JwtModule.registerAsync`.	Alta
S-04	RD-01 solo se aplica a códigos con "SINTOMA" o "DOLOR" en mayúsculas; el prompt enviado a Ollama no incluye la regla de no diagnosticar.	Una pregunta escrita libremente puede llegar al modelo sin bloqueo; `bloqueo_medico` siempre se guarda como `false`.	Agregar un filtro previo de palabras clave en el backend (sin distinguir mayúsculas), añadir la regla al prompt y registrar `bloqueo_medico: true`.	Alta
S-05	El PIN se genera con `Math.random()` y no expira.	PIN predecible o reutilizable si no se usa.	Usar `crypto.randomInt` y guardar fecha de expiración (p. ej. 10 min).	Media

ID	Hallazgo	Riesgo	Recomendación	Prioridad
S-06	Sin límite de intentos de inicio de sesión.	Ataques de fuerza bruta.	Agregar `@nestjs/throttler`.	Media
S-07	CORS abierto y túnel ngrok público.	Exposición del API a cualquier origen.	Restringir orígenes en producción y no usar ngrok fuera de desarrollo.	Media
S-08	La bitácora guarda el texto de las preguntas (posibles datos de salud).	Datos personales sensibles sin política de retención.	Definir retención (TTL) y aviso de privacidad conforme a la ley de protección de datos personales vigente.	Media
S-09	El token se guarda en AsyncStorage sin cifrar.	Lectura del token en un dispositivo comprometido.	Usar `expo-secure-store`.	Baja

11.3 Consideraciones éticas y legales

- El sistema maneja **datos personales sensibles de salud** (medicamentos, citas, síntomas mencionados). Debe contar con aviso de privacidad y consentimiento del titular.
- El asistente es una herramienta de **organización**, no un dispositivo médico: nunca debe sugerir diagnósticos, cambios de dosis ni sustituir la consulta profesional.
- El modo local con Ollama es la opción preferente cuando se requiera que la información no salga de la infraestructura propia.

12. Solución de problemas

La tabla reúne los errores más probables, con el mensaje tal como aparece en la app o en la consola, su causa y la solución.

Tabla 59. *Catálogo de errores y soluciones*

Código	Síntoma / mensaje	Causa probable	Solución
ERR-01	App: "No se pudo conectar con el servidor. Verifica tu red."	Backend apagado, túnel ngrok inactivo, URL distinta o con espacios en `baseURL`.	Levantar backend y ngrok; copiar la URL exacta en `api.client.ts` sin espacios; probar `lumaStatus` en el navegador.
ERR-02	App: "El servidor tardó demasiado en responder…"	Ollama cargando el modelo o equipo lento (más de 45 s).	Precargar con `ollama run llama3.2`; usar Luma Cloud; o aumentar `timeout`.
ERR-03	"Fallo en el motor de inferencia: connect ECONNREFUSED 127.0.0.1:8000"	El motor FastAPI no está corriendo.	Ejecutar `uvicorn app.main:app --port 8000` en `chatbot-lumaia`.
ERR-04	"…ECONNREFUSED 127.0.0.1:11434" o "model not found"	Ollama apagado o modelo no descargado.	`ollama serve` y `ollama pull llama3.2`.
ERR-05	"Field consultarAsistente of type AsistenteResponse! must have a selection of subfields"	La app pide un `String` y el backend devuelve un objeto.	En `ia.service.ts` pedir `{ respuesta_texto intencion }` y usar `respuesta_texto` en ChatScreen.
ERR-06	Backend: `MongooseServerSelectionError`	Servicio MongoDB local detenido, `MONGODB_URI` incorrecta o IP no permitida en Atlas.	Local: iniciar el servicio MongoDB y comprobar la conexión en Compass. Atlas: revisar la cadena y agregar la IP en *Network Access*.
ERR-07	"Correo o contraseña incorrectos." con usuarios del `seed.js`	El seed usa un hash ficticio (`$2b$10$hashedpasswordxyz`) que no corresponde a ninguna contraseña.	Registrar usuarios desde la app o reemplazar el hash por uno generado con bcrypt.
ERR-08	"Variable $input got invalid value 'Adulto Mayor'…"	Se envió el valor de la enumeración en lugar de su nombre GraphQL.	Enviar `ADULTO_MAYOR`/`CUIDADOR` usando `mapEnumToGraphQLRole`.
ERR-09	"El correo electrónico ya está registrado."	Índice único de `correo` (comportamiento esperado).	Usar otro correo o iniciar sesión.
ERR-10	Móvil: alerta "Audio Capturado… Transcriptor Whisper en desarrollo"	La transcripción de voz en móvil no está implementada.	Comportamiento esperado; escribir la pregunta o usar la versión web para dictado.
ERR-11	Tomas o citas de "hoy" no coinciden con el día real	El servidor calcula el día con su zona horaria local.	Definir `TZ=America/Mexico_City` en el servidor.
ERR-12	Expo: advertencias de versiones incompatibles	Paquetes instalados con `npm` en lugar de `expo install`.	`npx expo install --fix` y `npx expo-doctor`.
ERR-13	`npm run test:e2e` responde 404	`AppController` no está registrado en `AppModule`.	Reescribir la prueba para GraphQL o registrar el controlador.
ERR-14	El asistente muestra texto con llaves o comillas	Ollama no devolvió JSON válido.	Se usa el respaldo de `parsearRespuestaJSON`; reforzar el prompt o reducir la temperatura del modelo.
ERR-15	Error al compilar `bcrypt` en `npm install`	Módulo nativo sin binario precompilado para la versión de Node.	Usar Node 20 LTS y ejecutar `npm rebuild bcrypt`.

13. Limitaciones y trabajo futuro

13.1 Limitaciones actuales

- **Sin autorización por token en el servidor:** el JWT se emite y se envía, pero ningún resolver lo valida (S-01).
- **Restricción diagnóstica incompleta:** solo se detectan códigos con SINTOMA o DOLOR; el texto libre y el modo Ollama dependen del prompt (S-04, CP-IA-10).
- **Motor "Luma Cloud" basado en reglas:** reconoce cuatro códigos de pregunta; el texto libre recibe una respuesta genérica. El *system prompt* para un LLM en la nube está preparado, pero no conectado.
- **Voz a texto en móvil pendiente:** el audio se graba y se convierte a Base64, pero no se envía a ningún transcriptor (Whisper).
- **Sin notificaciones push ni escalamiento automático:** `tokens_dispositivo` existe, pero no hay integración con FCM/APNs ni tarea programada de 15 minutos (RF-20).
- **SOS limitado:** inicia una llamada al primer contacto; "Enviar Alerta" al cuidador es simulada.
- **Sin edición ni eliminación:** no existen mutaciones de actualización o borrado; el borrado lógico y el TTL están listos pero sin uso.
- **Vinculación y accesibilidad sin interfaz:** la API de PIN y el tema de alto contraste existen, pero no hay pantallas para usarlos.
- **Sin modo offline:** solo la sesión persiste; los datos requieren conexión.
- **Una toma por día:** `confirmarToma` no distingue entre los varios horarios de un medicamento, y el backend no impide confirmaciones duplicadas.
- **Dependencia de ngrok** y URLs fijas en el código para el motor IA y el backend.

13.2 Trabajo futuro

Tabla 60. *Hoja de ruta propuesta*

Prioridad	Mejora	Beneficio
1	Guard JWT, quitar `password_hash` del esquema y mover secretos a `.env`.	Cierra los hallazgos S-01 a S-03.
1	Filtro RD-01 en el backend para cualquier modo y texto, con registro de `bloqueo_medico`.	Cumplimiento ético total del asistente.
1	Corregir la consulta `consultarAsistente` en la app.	Restablece el chat (ERR-05).
2	Transcripción de voz (Whisper u otro STT) y uso de `evaluator.py` para la toma más próxima.	Interacción por voz completa y respuestas más precisas.
2	Notificaciones push (Expo Notifications/FCM) y tarea programada de escalamiento a 15 min.	Implementa RF-20 y la regla $(P \land \neg Q) \rightarrow R$.
2	Pantallas de vinculación por PIN, ajustes de accesibilidad y vista del cuidador.	Completa RF-04 y RF-14.

Prioridad	Mejora	Beneficio
3	Mutaciones de edición y borrado lógico; registro de toma por horario.	Gestión completa de la rutina y adherencia más exacta.
3	Caché persistente de React Query para modo offline.	Disponibilidad sin conexión.
2	Clasificador de intención por Distancia Euclidiana e indicador de adherencia por Distancia Manhattan (sección 3.9).	Luma entiende texto libre y el cuidador obtiene una métrica objetiva.
3	Escalamiento SOS con UCS y ajuste de recordatorios con Hill climbing (sección 3.9).	Alertas más eficaces una vez que existan notificaciones push.
3	Conectar un LLM en la nube con el system prompt existente y pruebas automatizadas de servicios.	Mejor comprensión del lenguaje natural y mayor calidad.

Referencias

Apollo Graph Inc. (2026). *Apollo Server documentation*. https://www.apollographql.com/docs/apollo-server

Expo. (2026). *Expo documentation (SDK 57)*. https://docs.expo.dev

Fundación OpenJS. (2026). *Node.js documentation*. https://nodejs.org/docs

GraphQL Foundation. (2021). *GraphQL specification*. https://spec.graphql.org

IEEE Computer Society. (1998). *IEEE recommended practice for software requirements specifications* (IEEE Std 830-1998). IEEE.

MongoDB, Inc. (2026). *Expire data from collections by setting TTL*. https://www.mongodb.com/docs/manual/core/index-ttl/

Mongoose. (2026). *Mongoose documentation*. https://mongoosejs.com/docs/

NestJS. (2026). *NestJS documentation: GraphQL, Mongo, Authentication*. https://docs.nestjs.com

Ollama. (2026). *Ollama API documentation*. https://github.com/ollama/ollama/blob/main/docs/api.md

Ramírez, S. (2026). *FastAPI documentation*. https://fastapi.tiangolo.com

Russell, S., y Norvig, P. (2021). *Artificial intelligence: A modern approach* (4.ª ed.). Pearson.

TanStack. (2026). *TanStack Query v5 documentation*. https://tanstack.com/query/latest

World Wide Web Consortium. (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Anexos

Anexo A. Esquema GraphQL

Contenido de `src/schema.gql` tal como está en el repositorio. Recuerde que el archivo se regenera al arrancar el backend; la mutación `consultarAsistente` aparecerá con tipo `AsistenteResponse!` en lugar de `String!`.

```
# ------------------------------------------------------
# THIS FILE WAS AUTOMATICALLY GENERATED (DO NOT MODIFY)
# ------------------------------------------------------

input AgregarContactoInput {
  nombre_contacto: String!
  parentesco: String!
  telefono: String!
  usuario_id: ID!
}

type Cita {
  _id: ID!
  estado: EnumEstadoCita!
  fecha_eliminacion: DateTime
  fecha_hora: DateTime!
  titulo_evento: String!
  ubicacion: String
  usuario_id: ID!
}

type ContactoEmergencia {
  nombre_contacto: String!
  parentesco: String!
  telefono: String!
}

input CreateCitaInput {
  estado: EnumEstadoCita
  fecha_hora: DateTime!
  titulo_evento: String!
  ubicacion: String
  usuario_id: ID!
}

input CreateMedicamentoInput {
  dosis: String!
  frecuencia_horas: Int
  horarios_especificos: [String!]! = []
  nombre_farmaco: String!
  usuario_id: ID!
}

input CreateNotaPersonalInput {
  contenido: String!
  fecha_creacion: DateTime!
  usuario_id: ID!
}

input CreateUsuarioInput {
  correo: String!
  nombre: String!
  password: String!
  rol: Enum_Rol!
}

"""
A date-time string at UTC, such as 2019-12-03T09:54:33Z, compliant with the date-time format.
```

```
"""
scalar DateTime

enum EnumEstadoCita {
  CANCELADA
  COMPLETADA
  PROGRAMADA
}

enum Enum_EstadoToma {
  CONFIRMADA
  OMITIDA
  PENDIENTE
}

enum Enum_Rol {
  ADULTO_MAYOR
  CUIDADOR
}

type LogInteraccion {
  _id: ID!
  bloqueo_medico: Boolean!
  fecha_interaccion: DateTime!
  premisa_entrada: String!
  respuesta_generada: String!
  usuario_id: ID!
}

type LoginResponse {
  access_token: String!
  rol: Enum_Rol!
  usuario_id: ID!
}

type Medicamento {
  _id: ID!
  dosis: String!
  fecha_eliminacion: DateTime
  frecuencia_horas: Int
  horarios_especificos: [String!]!
  nombre_farmaco: String!
  usuario_id: ID!
}

type Mutation {
  agregarContactoEmergencia(input: AgregarContactoInput!): Usuario!
  confirmarTomaMedicamento(medicamento_id: ID!): RegistroToma!
  consultarAsistente(modelo: String, pregunta_id: String!, usuario_id: ID!): String!
  crearCita(createCitaInput: CreateCitaInput!): Cita!
  crearMedicamento(createMedicamentoInput: CreateMedicamentoInput!): Medicamento!
  crearNotaPersonal(createNotaPersonalInput: CreateNotaPersonalInput!): NotaPersonal!
  generarPinVinculacion(usuario_id: ID!): String!
  login(correo: String!, password: String!): LoginResponse!
  registrarUsuario(input: CreateUsuarioInput!): Usuario!
  vincularCuidador(input: VincularCuidadorInput!): Usuario!
}

type NotaPersonal {
  _id: ID!
  contenido: String!
  fecha_creacion: DateTime!
  fecha_eliminacion: DateTime
  usuario_id: ID!
}

type Query {
  logsInteraccionBase: [LogInteraccion!]!

  """Estado general del API LumaIA"""
```

```
  lumaStatus: String!
  obtenerCitasPorUsuario(usuario_id: ID!): [Cita!]!
  obtenerMedicamentosPorUsuario(usuario_id: ID!): [Medicamento!]!
  obtenerNotasPersonalesPorUsuario(usuario_id: ID!): [NotaPersonal!]!
  obtenerTomasHoy(usuario_id: ID!): [RegistroToma!]!
  obtenerUsuario(usuario_id: ID!): Usuario!
  registrosTomaBase: [RegistroToma!]!
}

type RegistroToma {
  _id: ID!
  estado_toma: Enum_EstadoToma!
  fecha_confirmacion: DateTime
  fecha_programada: DateTime!
  medicamento_id: ID!
}

type Usuario {
  _id: ID!
  contactos_emergencia: [ContactoEmergencia!]!
  correo: String!
  cuidador_vinculado_id: ID
  estado_activo: Boolean!
  nombre: String!
  password_hash: String!
  pin_vinculacion: String
  rol: Enum_Rol!
  tokens_dispositivo: [String!]!
}

input VincularCuidadorInput {
  cuidador_id: ID!
  pin: String!
}
```

Anexo B. Datos de prueba (database/seed.js)

Tabla 61. *Documentos generados por el script de datos de prueba*

Colección	Documentos
usuarios	Pedro (Adulto Mayor, `pedro@lumaia.local`, vinculado a Osvaldo, contactos: Osvaldo y Yeni); Osvaldo Salinas Aranda (Cuidador); Jaredtzi Aide (Cuidador)
medicamentos	Losartán, 1 pastilla, cada 12 h, 08:00 y 20:00; Metformina, 850 mg, cada 24 h, 14:00
citas	Revisión General, 15/10/2026 10:00 UTC, Clínica del Centro, Consultorio 4
notas_personales	"Preguntar al doctor sobre el cambio de marca del medicamento."
registros_toma	Losartán confirmada (29/09/2026); Metformina pendiente (29/09/2026)
logs_interaccion	Consulta de horario (`bloqueo_medico: false`); consulta de síntoma (`bloqueo_medico: true`)

Anexo C. Guía rápida de comandos

Tabla 62. *Comandos más usados*

Acción	Comando
Backend en desarrollo	`npm run start:dev`
Backend compilado	`npm run build && npm run start:prod`
Motor IA	`uvicorn app.main:app --port 8000 --reload`
App móvil	`npx expo start` (tecla `a` Android, `w` web)
Túnel	`ngrok http 3000`
Datos de prueba	Consola mongosh de Compass o `mongosh "<MONGODB_URI>" database/seed.js`
Formato y lint	`npm run format && npm run lint`
Compilar APK de prueba	`npx eas-cli@latest build -p android --profile preview`

Lista de verificación antes de entregar

Marque cada punto antes de entregar el manual o liberar una nueva versión del sistema.

✓	Ámbito	Punto de control
☐	Documento	Portada con institución, asignatura, docente, integrantes, grupo, lugar y fecha
☐	Documento	Control de versiones actualizado con la versión entregada
☐	Documento	Índice automático actualizado (en Word: clic derecho › Actualizar campo › Actualizar toda la tabla)
☐	Documento	Las 13 secciones solicitadas están presentes y en orden
☐	Documento	Tablas numeradas, con título y legibles (esta versión no incluye diagramas)
☐	Documento	Las cuatro tablas de verdad (general, R1, R2 y R3) están después del marco PEAS
☐	Documento	Glosario, referencias en formato APA y anexos incluidos
☐	Documento	Revisión ortográfica y de nombres de integrantes
☐	Documento	Archivo entregado en .docx o exportado a .pdf
☐	Sistema	La base `lumaia` es accesible en Compass (local) y en Atlas (nube)
☐	Sistema	El backend arranca sin errores y `lumaStatus` responde
☐	Sistema	El motor IA responde en `/evaluar` y Ollama tiene `llama3.2`
☐	Sistema	`baseURL` de la app apunta a la URL vigente, sin espacios
☐	Sistema	El archivo `.env` existe localmente y no está en el repositorio
☐	Sistema	Los casos de prueba de la sección 9 se ejecutaron y sus resultados coinciden
☐	Sistema	`npm run lint` y `npx tsc --noEmit` sin errores
☐	Sistema	Los hallazgos de seguridad de prioridad alta están atendidos o documentados